（收录范围：examdiag/ 核心包 + tests/ 测试 + scripts/ 工具脚本，全部真实源码）

```
# ============ 文件: examdiag/__init__.py (3 行) ============
"""exam-diag: find the root-cause concept behind every wrong answer."""

__version__ = "0.1.0"

# ============ 文件: examdiag/schema.py (185 行) ============
"""Core data models: Concept / Question / Attempt / ConceptGraph.

中文文本字段一律 UTF-8 str。prereqs 引用校验在 ConceptGraph 里做：
未知前置 id 或成环都会抛 ValueError，环路报错里带具体路径。
"""

from __future__ import annotations

import json
from collections import deque
from datetime import datetime
from pathlib import Path
from typing import Literal

import yaml
from pydantic import BaseModel, Field, field_validator, model_validator

Subject = Literal["408", "math1", "english", "politics"]


class Concept(BaseModel):
    """知识点节点；id 用点分层级（如 ds.bst.rotate），prereqs 指向前置概念 id。"""

    id: str = Field(pattern=r"^[a-z][a-z0-9]*(\.[a-z0-9_-]+)+$")
    name: str = Field(min_length=1)
    subject: Subject
    difficulty: float = Field(ge=1.0, le=5.0)
    prereqs: list[str] = Field(default_factory=list)

    @field_validator("subject", mode="before")
    @classmethod
    def _coerce_subject(cls, v: object) -> object:
        # YAML 会把未加引号的 408 解析成 int，这里拉回字符串
        return str(v) if isinstance(v, int) else v


class Question(BaseModel):
    """题目；concepts 里主概念排第一，source 用真题出处或 self-built。"""

    id: str = Field(min_length=1)
    subject: Subject
    stem: str = Field(min_length=1)
    answer: str = Field(min_length=1)
    concepts: list[str] = Field(min_length=1)
```

```
    source: str = Field(min_length=1)

    @field_validator("subject", mode="before")
    @classmethod
    def _coerce_subject(cls, v: object) -> object:
        return str(v) if isinstance(v, int) else v


class Attempt(BaseModel):
    """一次作答记录；ts 需要能排序，BKT 按序列顺序消费。"""

    user_id: str = Field(min_length=1)
    question_id: str = Field(min_length=1)
    correct: bool
    ts: datetime


class ConceptGraph(BaseModel):
    """一批概念的集合，加载时校验 prereq 引用与无环性。"""

    concepts: dict[str, Concept]

    @classmethod
    def from_list(cls, items: list[Concept]) -> ConceptGraph:
        return cls(concepts={c.id: c for c in items})

    @model_validator(mode="after")
    def _check_references_and_cycles(self) -> ConceptGraph:
        self.check_prereq_ids()
        self.check_acyclic()
        return self

    def check_prereq_ids(self) -> None:
        unknown = [
            (c.id, p)
            for c in self.concepts.values()
            for p in c.prereqs
            if p not in self.concepts
        ]
        if unknown:
            sample = ", ".join(f"{cid}->{p}" for cid, p in unknown[:5])
            raise ValueError(f"unknown prereq id(s) ({len(unknown)}): {sample}")

    def find_cycle(self) -> list[str] | None:
        """迭代式 DFS 找环，返回环路路径（首尾同点）；无环返回 None。"""
        WHITE, GRAY, BLACK = 0, 1, 2
        color = dict.fromkeys(self.concepts, WHITE)
        for start in self.concepts:
            if color[start] != WHITE:
                continue
```

```
            color[start] = GRAY
            stack = [(start, iter(self.concepts[start].prereqs))]
            path = [start]
            while stack:
                node, it = stack[-1]
                advanced = False
                for nxt in it:
                    if color[nxt] == GRAY:
                        return path[path.index(nxt):] + [nxt]
                    if color[nxt] == WHITE:
                        color[nxt] = GRAY
                        stack.append((nxt, iter(self.concepts[nxt].prereqs)))
                        path.append(nxt)
                        advanced = True
                        break
                if not advanced:
                    color[node] = BLACK
                    stack.pop()
                    path.pop()
        return None

    def check_acyclic(self) -> None:
        cycle = self.find_cycle()
        if cycle:
            raise ValueError("prereq cycle detected: " + " -> ".join(cycle))

    def roots(self) -> list[str]:
        """无前置的节点 id，保持声明顺序。"""
        return [cid for cid, c in self.concepts.items() if not c.prereqs]

    def topo_order(self) -> list[str]:
        """Kahn 拓扑序（同层按 id 排序保证确定性）；有环时抛 ValueError。"""
        indeg = dict.fromkeys(self.concepts, 0)
        dependents: dict[str, list[str]] = {cid: [] for cid in self.concepts}
        for c in self.concepts.values():
            for p in c.prereqs:
                indeg[c.id] += 1
                dependents[p].append(c.id)
        queue = deque(sorted(cid for cid, d in indeg.items() if d == 0))
        order: list[str] = []
        while queue:
            n = queue.popleft()
            order.append(n)
            for m in sorted(dependents[n]):
                indeg[m] -= 1
                if indeg[m] == 0:
                    queue.append(m)
        if len(order) != len(self.concepts):
            raise ValueError("graph has a cycle; topo order undefined")
        return order
```

```
151
152     def ancestors(self, cid: str) -> set[str]:
153         """cid 的全部传递前置（不含自身）。"""
154         seen: set[str] = set()
155         stack = list(self.concepts[cid].prereqs)
156         while stack:
157             p = stack.pop()
158             if p in seen:
159                 continue
160             seen.add(p)
161             stack.extend(self.concepts[p].prereqs)
162         return seen
163
164
165 # ---------- 文件加载 ----------
166
167
168 def load_concepts(path: str | Path) -> ConceptGraph:
169     """加载科目概念文件；接受 {concepts: [...]} 或裸列表两种 YAML 形状。"""
170     with open(path, encoding="utf-8") as f:
171         raw = yaml.safe_load(f)
172     items = raw["concepts"] if isinstance(raw, dict) else raw
173     return ConceptGraph.from_list([Concept(**item) for item in items])
174
175
176 def load_questions(path: str | Path) -> list[Question]:
177     out: list[Question] = []
178     with open(path, encoding="utf-8") as f:
179         for line in f:
180             if line.strip():
181                 out.append(Question(**json.loads(line)))
182     return out
183
184
185 def load_attempts(path: str | Path) -> list[Attempt]:
186     out: list[Attempt] = []
187     with open(path, encoding="utf-8") as f:
188         for line in f:
189             if line.strip():
190                 out.append(Attempt(**json.loads(line)))
191     return out
192
193 # ============ 文件: examdiag/bkt.py  (192 行) ============
194 """Bayesian Knowledge Tracing: 前向后验更新 + EM 拟合（纯 numpy）。
195
196 模型（每个 concept 独立一套参数）：
197   状态：L=已掌握，U=未掌握；P(L_1)=p_L0
198   转移：U->L 概率 p_T（learn），L 不会退回 U
199   发射：P(correct|L)=1-p_S（slip），P(correct|U)=p_G（guess）
200
```

```
推导见 docs/METHOD.md (Wave 3 补全) 。实现细节：
  - 前向/后向都做 per-step 归一化 (scaled Baum-Welch) ，防止长序列下溢；
    缩放常数对数之和即 log-likelihood。
  - EM 的 M 步为闭式重估，四参数每轮截断到 [0.01, 0.99]。
  - fit 支持变长序列：按长度分组批量算充分统计量，合并后统一 M 步。
"""

from __future__ import annotations

from collections import defaultdict
from dataclasses import dataclass

import numpy as np

PARAM_LOW, PARAM_HIGH = 0.01, 0.99


@dataclass(frozen=True)
class BKTParams:
    p_L0: float
    p_T: float
    p_S: float
    p_G: float

    def clipped(self) -> BKTParams:
        return BKTParams(*(float(np.clip(v, PARAM_LOW, PARAM_HIGH)) for v in self.as_array()))

    def as_array(self) -> np.ndarray:
        return np.array([self.p_L0, self.p_T, self.p_S, self.p_G], dtype=float)

    @classmethod
    def from_array(cls, arr) -> BKTParams:
        return cls(*[float(x) for x in arr])


def _check_sequence(seq) -> np.ndarray:
    obs = np.asarray(seq, dtype=bool)
    if obs.ndim != 1:
        raise ValueError(f"sequence must be 1-D 0/1, got shape {obs.shape}")
    return obs


def forward_posterior(sequence, params: BKTParams) -> np.ndarray:
    """每一步的过滤后验 P(L_t | o_1..o_t)，长度与输入相同。"""
    obs = _check_sequence(sequence)
    b_L = np.where(obs, 1.0 - params.p_S, params.p_S)
    b_U = np.where(obs, params.p_G, 1.0 - params.p_G)

    post = np.empty(obs.shape[0])
    p = params.p_L0  # P(L_t) 进入第 t 次观测前的掌握概率
```

```
    for t in range(obs.shape[0]):
        a_L = p * b_L[t]
        a_U = (1.0 - p) * b_U[t]
        post[t] = p = a_L / (a_L + a_U)  # 证据更新
        p = post[t] + (1.0 - post[t]) * params.p_T  # 练习转移，供下一步发射
    return post


def predict_next(sequence, params: BKTParams) -> float:
    """P(下一题答对 | 已观察序列)：先转移 (learn) 再发射。"""
    obs = _check_sequence(sequence)
    p = params.p_L0 if obs.size == 0 else float(forward_posterior(obs, params)[-1])
    p_next = p + (1.0 - p) * params.p_T
    return float(p_next * (1.0 - params.p_S) + (1.0 - p_next) * params.p_G)


# ---------- EM (Baum-Welch)  ----------


def _group_by_length(sequences) -> dict[int, np.ndarray]:
    groups: dict[int, list] = defaultdict(list)
    for seq in sequences:
        groups[len(seq)].append(_check_sequence(seq))
    return {n: np.stack(seqs) for n, seqs in groups.items()}


def _group_stats(obs: np.ndarray, p: BKTParams) -> dict[str, float]:
    """一组等长序列 (n, T) 的充分统计量。obs 为 bool 矩阵，1=True=答对。"""
    n, T = obs.shape
    p_L0, p_T, p_S, p_G = p.p_L0, p.p_T, p.p_S, p.p_G

    b_L = np.where(obs, 1.0 - p_S, p_S)  # 发射：已掌握
    b_U = np.where(obs, p_G, 1.0 - p_G)  # 发射：未掌握

    # 前向 (缩放)
    alpha_L = np.empty((n, T))
    alpha_U = np.empty((n, T))
    c = np.empty((n, T))
    alpha_L[:, 0] = p_L0 * b_L[:, 0]
    alpha_U[:, 0] = (1.0 - p_L0) * b_U[:, 0]
    c[:, 0] = alpha_L[:, 0] + alpha_U[:, 0]
    alpha_L[:, 0] /= c[:, 0]
    alpha_U[:, 0] /= c[:, 0]
    for t in range(1, T):
        alpha_L[:, t] = (alpha_L[:, t - 1] + p_T * alpha_U[:, t - 1]) * b_L[:, t]
        alpha_U[:, t] = (1.0 - p_T) * alpha_U[:, t - 1] * b_U[:, t]
        c[:, t] = alpha_L[:, t] + alpha_U[:, t]
        alpha_L[:, t] /= c[:, t]
        alpha_U[:, t] /= c[:, t]

```

```
    loglik = float(np.log(c).sum())

    # 后向（与前向同一套缩放常数）
    beta_L = np.ones((n, T))
    beta_U = np.ones((n, T))
    for t in range(T - 2, -1, -1):
        beta_L[:, t] = beta_L[:, t + 1] * b_L[:, t + 1] / c[:, t + 1]
        beta_U[:, t] = (
            p_T * beta_L[:, t + 1] * b_L[:, t + 1]
            + (1.0 - p_T) * beta_U[:, t + 1] * b_U[:, t + 1]
        ) / c[:, t + 1]

    gamma_L = alpha_L * beta_L
    gamma_U = alpha_U * beta_U

    # 期望转移数 ξ(U->L) = α_U(t)·p_T·b_L(t+1)·β_L(t+1)/c(t+1)，t 只取 1..T-1
    if T > 1:
        xi_UL = float((alpha_U[:, :-1] * p_T * b_L[:, 1:] * beta_L[:, 1:] / c[:, 1:]).sum())
        gamma_U_pre = float(gamma_U[:, :-1].sum())
    else:
        xi_UL, gamma_U_pre = 0.0, 0.0

    return {
        "n": n,
        "loglik": loglik,
        "gamma_L0": float(gamma_L[:, 0].sum()),
        "N_L": float(gamma_L.sum()),
        "N_U": float(gamma_U.sum()),
        "N_L_wrong": float(gamma_L[:, :][~obs].sum()),
        "N_U_correct": float(gamma_U[:, :][obs].sum()),
        "xi_UL": xi_UL,
        "gamma_U_pre": gamma_U_pre,
    }


def _m_step(stats: dict[str, float], old: BKTParams) -> BKTParams:
    n_total = stats["n"]
    p_L0 = stats["gamma_L0"] / n_total
    p_S = stats["N_L_wrong"] / stats["N_L"] if stats["N_L"] > 0 else old.p_S
    p_G = stats["N_U_correct"] / stats["N_U"] if stats["N_U"] > 0 else old.p_G
    p_T = stats["xi_UL"] / stats["gamma_U_pre"] if stats["gamma_U_pre"] > 0 else old.p_T
    return BKTParams(p_L0, p_T, p_S, p_G).clipped()


def fit(
    sequences,
    init: BKTParams | None = None,
    max_iter: int = 200,
    tol: float = 1e-6,
) -> BKTParams:
```

```
    """EM 拟合四参数；log-likelihood 相对变化 < tol 或达到 max_iter 停止。"""
    groups = _group_by_length(sequences)
    if not groups:
        raise ValueError("no sequences to fit")
    params = init or BKTParams(p_L0=0.4, p_T=0.15, p_S=0.15, p_G=0.15)
    prev_ll = -np.inf
    for _ in range(max_iter):
        acc = {"n": 0.0, "loglik": 0.0, "gamma_L0": 0.0, "N_L": 0.0, "N_U": 0.0,
               "N_L_wrong": 0.0, "N_U_correct": 0.0, "xi_UL": 0.0, "gamma_U_pre": 0.0}
        for obs in groups.values():
            for key, val in _group_stats(obs, params).items():
                acc[key] += val
        loglik = acc["loglik"]
        params = _m_step(acc, params)
        if abs(loglik - prev_ll) < tol * max(1.0, abs(loglik)):
            break
        prev_ll = loglik
    return params


def log_likelihood(sequences, params: BKTParams) -> float:
    groups = _group_by_length(sequences)
    return float(sum(_group_stats(obs, params)["loglik"] for obs in groups.values()))


def simulate(params: BKTParams, n_students: int, seq_len: int, seed: int = 0) -> np.ndarray:
    """按 BKT 生成 (n_students, seq_len) 的 0/1 作答矩阵，用于自检与评估。"""
    rng = np.random.default_rng(seed)
    obs = np.empty((n_students, seq_len), dtype=bool)
    learned = rng.random(n_students) < params.p_L0
    for t in range(seq_len):
        p_correct = np.where(learned, 1.0 - params.p_S, params.p_G)
        obs[:, t] = rng.random(n_students) < p_correct
        learned = learned | (rng.random(n_students) < params.p_T)
    return obs

# ============ 文件: examdiag/graph.py  (97 行) ============
"""DAG 操作层 (networkx) ：拓扑序、祖先闭包、最短路径、环检测。

图方向统一为 前置概念 -> 后继概念 (prereq -> concept) ，
因此 nx.ancestors(g, c) 即 c 的全部前置，nx.descendants(g, c) 即依赖 c 的概念。
schema.ConceptGraph 在加载时已做环检测；本模块对已构建的 DiGraph 再暴露
独立环检测，供 Wave 1 的诊断流程与测试复用。
"""

from __future__ import annotations

from collections.abc import Iterable, Iterator

import networkx as nx
```

```python

from .schema import ConceptGraph


def build(concepts: ConceptGraph) -> nx.DiGraph:
    """从 ConceptGraph 构建 networkx 有向图（前置 -> 后继），节点带 name/difficulty 属性。"""
    g = nx.DiGraph()
    for cid, c in concepts.concepts.items():
        g.add_node(cid, name=c.name, difficulty=c.difficulty, subject=c.subject)
    for cid, c in concepts.concepts.items():
        for p in c.prereqs:
            g.add_edge(p, cid)
    return g


def detect_cycle(g: nx.DiGraph) -> list[str] | None:
    """返回一条环路路径（首尾同点）；无环返回 None。"""
    try:
        cycle = nx.find_cycle(g, orientation="original")
    except nx.NetworkXNoCycle:
        return None
    # find_cycle 返回边列表 [(u, v, dir), ...]，串成节点路径
    path = [cycle[0][0]]
    for _u, v, _d in cycle:
        path.append(v)
    return path


def topo_order(g: nx.DiGraph) -> list[str]:
    """拓扑序（networkx lexicographical 变体保证同层确定性）。"""
    return list(nx.lexicographical_topological_sort(g))


def ancestors(g: nx.DiGraph, cid: str) -> set[str]:
    return nx.ancestors(g, cid)


def descendants(g: nx.DiGraph, cid: str) -> set[str]:
    return nx.descendants(g, cid)


def ancestor_closure(g: nx.DiGraph, cids: Iterable[str]) -> set[str]:
    """cids 及其全部传递前置。"""
    ids = set(cids)
    missing = ids - set(g.nodes)
    if missing:
        raise KeyError(f"unknown concept id(s): {sorted(missing)}")
    closure: set[str] = set(ids)
    for cid in ids:
        closure |= nx.ancestors(g, cid)
```

```
    return closure


def subgraph(g: nx.DiGraph, nodes: Iterable[str]) -> nx.DiGraph:
    return g.subgraph(set(nodes)).copy()


def distance(g: nx.DiGraph, src: str, dst: str) -> int:
    """沿前置方向 src -> dst 的最短距离；不可达抛 ValueError。"""
    try:
        return nx.shortest_path_length(g, src, dst)
    except nx.NetworkXNoPath as e:
        raise ValueError(f"no path from {src} to {dst}") from e


def shortest_path(g: nx.DiGraph, src: str, dst: str) -> list[str]:
    """src -> dst 最短路径上的概念序列（含两端）。"""
    try:
        return nx.shortest_path(g, src, dst)
    except nx.NetworkXNoPath as e:
        raise ValueError(f"no path from {src} to {dst}") from e


def iter_reachable_targets(
    g: nx.DiGraph, src: str, targets: Iterable[str]
) -> Iterator[tuple[str, int]]:
    """依次产出 (target, distance)：src 可达的 targets，按距离升序、id 升序。"""
    out = []
    for t in targets:
        try:
            out.append((t, nx.shortest_path_length(g, src, t)))
        except nx.NetworkXNoPath:
            continue
    return iter(sorted(out, key=lambda x: (x[1], x[0])))

# ============ 文件: examdiag/diagnose.py  (218 行) ============
"""根因诊断主算法 (SPEC 5.3) 。

流程：
1. attempts 按时间排序，每次作答归属到题目挂载的每个概念 → 每概念一条 0/1 序列
2. 概念级 BKT 拟合 → mastery[c] = 最后一次作答后的过滤后验
3. 错题概念集 C_w 及其祖先闭包为候选集；按 SPEC 公式计算
   score(c) = (1-mastery[c]) * Π_{a∈anc(c)} (1-mastery[a])^(1/d(a,c))
   （无数据的前置不进乘积；指数 1/d 让远前置权重衰减）
4. 症状过滤：候选若有掌握度 < MASTERY_THRESHOLD 的有数据前置，则其失败
   可被上游解释，标记为症状而非根因—即 SPEC 第 4 步"祖先闭包内取最上游
   低掌握节点"的实现形式
5. 幸存者按 mastery 升序 + 拓扑序取前 top_n 个为根因；每个根因附：
   涉及错题数（自身或其下游的错题）、到最近错题概念的图距离、
   补习路径（根因 -> 最近错题概念的最短路径）、它解释的下游症状
```

```
501
502 边界：概念 attempt 数 < MIN_ATTEMPTS(3) 时不参与拟合与根因候选、不进 score
503 乘积，标记为 insufficient_data。
504 """
505
506 from __future__ import annotations
507
508 from collections import Counter, defaultdict
509 from collections.abc import Sequence
510 from dataclasses import dataclass, field
511
512 import networkx as nx
513 import numpy as np
514
515 from .bkt import BKTParams, fit, forward_posterior
516 from .graph import (
517     ancestor_closure,
518     ancestors,
519     build,
520     descendants,
521     distance,
522     shortest_path,
523     topo_order,
524 )
525 from .schema import Attempt, ConceptGraph, Question
526
527 MIN_ATTEMPTS = 3
528 MASTERY_THRESHOLD = 0.5  # 低于它视为"未掌握"，用于症状过滤
529 DEGENERATE_P = 0.5  # p_S/p_G 超过它说明 EM 在小样本上退化了
530 DEFAULT_INIT = BKTParams(p_L0=0.4, p_T=0.15, p_S=0.15, p_G=0.15)
531
532
533 @dataclass
534 class RootCause:
535     concept_id: str
536     name: str
537     mastery: float
538     score: float
539     wrong_count: int
540     min_distance: int
541     remedy_path: list[str] = field(default_factory=list)
542     explains: list[str] = field(default_factory=list)
543
544     def to_dict(self) -> dict:
545         return {
546             "concept_id": self.concept_id,
547             "name": self.name,
548             "mastery": round(self.mastery, 4),
549             "score": round(self.score, 6),
550             "wrong_count": self.wrong_count,
```

```
            "min_distance": self.min_distance,
            "remedy_path": self.remedy_path,
            "explains": self.explains,
        }


@dataclass
class Diagnosis:
    user_id: str | None
    root_causes: list[RootCause]
    insufficient_data: list[str]
    mastery: dict[str, float]

    def to_dict(self) -> dict:
        return {
            "user_id": self.user_id,
            "root_causes": [r.to_dict() for r in self.root_causes],
            "insufficient_data": self.insufficient_data,
            "mastery": {k: round(v, 4) for k, v in sorted(self.mastery.items())},
        }


def collect_sequences(
    attempts: Sequence[Attempt], questions: Sequence[Question], user_id: str | None = None
) -> dict[str, list[int]]:
    """按概念归并作答序列：每题归属到它挂载的每个概念，序列按 ts 升序。"""
    qmap = {q.id: q for q in questions}
    rows = [a for a in attempts if user_id is None or a.user_id == user_id]
    if user_id is not None and not rows:
        raise KeyError(f"no attempts for user {user_id!r}")
    rows.sort(key=lambda a: a.ts)
    seqs: dict[str, list[int]] = defaultdict(list)
    for a in rows:
        q = qmap.get(a.question_id)
        if q is None:
            raise KeyError(f"attempt references unknown question {a.question_id!r}")
        for cid in q.concepts:
            seqs[cid].append(int(a.correct))
    return dict(seqs)


def _mastery_of(obs: np.ndarray) -> float:
    """单概念掌握度：EM 拟合后取末次作答的过滤后验。

    全对/全错等极短序列会让 EM 退化（如全错序列把 p_S 顶到上界 0.99，
    拟合成"已掌握但总失误"，与"未掌握"似然相同而后验相反）。检测到
    p_S/p_G 越过 0.5 即视为退化，退回默认参数做前向后验，保证全对→高、
    全错→低的直觉排序。
    """
    params = fit([obs])
```

```
    if params.p_S > DEGENERATE_P or params.p_G > DEGENERATE_P:
        params = DEFAULT_INIT
    return float(forward_posterior(obs, params)[-1])


def estimate_mastery(seqs: dict[str, list[int]]) -> tuple[dict[str, float], list[str]]:
    """逐概念拟合给出末次作答后的掌握后验；样本不足的进 insufficient。"""
    mastery: dict[str, float] = {}
    insufficient: list[str] = []
    for cid, seq in sorted(seqs.items()):
        if len(seq) < MIN_ATTEMPTS:
            insufficient.append(cid)
            continue
        mastery[cid] = _mastery_of(np.asarray(seq, dtype=bool))
    return mastery, sorted(insufficient)


def score(g: nx.DiGraph, cid: str, mastery: dict[str, float]) -> float:
    """SPEC 5.3 公式：(1-mastery[c]) * Π (1-mastery[a])^(1/d(a,c))，无数据前置跳过。"""
    prod = 1.0
    for a in ancestors(g, cid):
        if a not in mastery:
            continue
        d = distance(g, a, cid)
        prod *= (1.0 - mastery[a]) ** (1.0 / d)
    return (1.0 - mastery[cid]) * prod


def diagnose(
    attempts: Sequence[Attempt],
    questions: Sequence[Question],
    concepts: ConceptGraph,
    user_id: str | None = None,
    top_n: int = 3,
) -> Diagnosis:
    g = build(concepts)
    qmap = {q.id: q for q in questions}
    seqs = collect_sequences(attempts, questions, user_id)
    mastery, insufficient = estimate_mastery(seqs)

    wrong_by_concept: Counter[str] = Counter()
    for a in attempts:
        if user_id is not None and a.user_id != user_id:
            continue
        if a.correct:
            continue
        for cid in qmap[a.question_id].concepts:
            wrong_by_concept[cid] += 1
    wrong_concepts = set(wrong_by_concept)
    if not wrong_concepts:
```

```
        return Diagnosis(
            user_id=user_id, root_causes=[], insufficient_data=insufficient, mastery=mastery
        )

    closure = ancestor_closure(g, wrong_concepts)
    candidates = {c for c in closure if c in mastery}
    topo_idx = {cid: i for i, cid in enumerate(topo_order(g))}

    # 症状过滤：有未掌握前置的概念是症状，不是根因；根因自身也必须未掌握
    survivors = [
        c
        for c in candidates
        if mastery[c] < MASTERY_THRESHOLD
        and not any(
            a in mastery and mastery[a] < MASTERY_THRESHOLD for a in ancestors(g, c)
        )
    ]
    survivors.sort(key=lambda c: (mastery[c], topo_idx[c]))

    root_causes: list[RootCause] = []
    for r in survivors[:top_n]:
        # 根因必然可达至少一个错题概念（r ∈ C_w 或其祖先）；跨分支的不可达概念跳过
        reachable = []
        for w in wrong_concepts:
            try:
                reachable.append((distance(g, r, w), topo_idx[w], w))
            except ValueError:
                continue
        min_dist = min(d for d, _, _ in reachable)
        # 补习终点取可达的最深错题概念：从根因一路补到最终要攻克的目标
        _, _, goal = max(reachable)
        desc = descendants(g, r)
        root_causes.append(
            RootCause(
                concept_id=r,
                name=concepts.concepts[r].name,
                mastery=mastery[r],
                score=score(g, r, mastery),
                wrong_count=sum(
                    cnt for cid, cnt in wrong_by_concept.items() if cid == r or cid in desc
                ),
                min_distance=min_dist,
                remedy_path=shortest_path(g, r, goal),
                explains=sorted(
                    (w for w in wrong_concepts if w in desc), key=topo_idx.__getitem__
                ),
            )
        )
    return Diagnosis(
        user_id=user_id,
```

```
        root_causes=root_causes,
        insufficient_data=insufficient,
        mastery=mastery,
    )

# ============ 文件: examdiag/srs.py (45 行) ============
"""间隔重复复习调度 (Wave 1 任务 1.3) 。

按根因掌握度生成未来 7 天的每日复习序列:
- 第 1 天复习全部根因,同一天内掌握度最低的排最前(最弱先补);
- 之后按间隔 repeats:interval = max(1, round(SPAN * mastery)),
  掌握度越低间隔越短、7 天内出现越频繁("间隔按 mastery 反比"的离散实现) 。
"""

from __future__ import annotations

from collections import defaultdict
from dataclasses import dataclass, field

from .diagnose import Diagnosis

SPAN = 6.0  # interval = max(1, round(SPAN * mastery)),根因 mastery < 0.5 → interval <= 3


@dataclass
class DayPlan:
    day: int
    concepts: list[str] = field(default_factory=list)

    def to_dict(self) -> dict:
        return {"day": self.day, "concepts": self.concepts}


def plan_week(diagnosis: Diagnosis, horizon: int = 7, span: float = SPAN) -> list[DayPlan]:
    schedule: dict[int, set[str]] = defaultdict(set)
    for rc in diagnosis.root_causes:
        interval = max(1, round(span * rc.mastery))
        day = 1
        while day <= horizon:
            schedule[day].add(rc.concept_id)
            day += interval
    plans = []
    for day in range(1, horizon + 1):
        ids = schedule.get(day, set())
        plans.append(
            DayPlan(
                day=day,
                concepts=sorted(ids, key=lambda cid: (diagnosis.mastery.get(cid, 0.0), cid)),
            )
        )
```

```
    return plans

# ============ 文件: examdiag/eval.py (167 行) ============
"""Next-response prediction benchmark: AUC / RMSE.

协议 (SPEC 5.4) : 合成多概念答题序列，每条序列前 train_len 次作答用于拟合
(按概念聚合 fit EM) ，余下部分做在线预测—每一步在观测前给出 P(答对)，
观测后把该次作答计入后验继续走。不依赖 scikit-learn，AUC 用带并列秩的
Mann-Whitney U 实现。

ASSISTments 真实数据集的下载与比对留给后续 Wave；本文件按 SPEC 允许的
路径用仓库内合成序列 (seed 固定，可复现) 跑通基线。
"""

from __future__ import annotations

from pathlib import Path

import numpy as np

from .bkt import BKTParams, fit, predict_next

DEFAULT_TRAIN_LEN = 8
DEFAULT_SEQ_LEN = 12


def auc_score(y_true, y_score) -> float:
    """带并列平均秩的 AUC (Mann-Whitney U 统计量的归一化形式) 。"""
    y_true = np.asarray(y_true, dtype=bool)
    y_score = np.asarray(y_score, dtype=float)
    n_pos = int(y_true.sum())
    n_neg = y_true.size - n_pos
    if n_pos == 0 or n_neg == 0:
        raise ValueError("AUC undefined when only one class is present")
    order = np.argsort(y_score, kind="mergesort")
    sx = y_score[order]
    ranks = np.empty(sx.size)
    i = 0
    while i < sx.size:
        j = i
        while j + 1 < sx.size and sx[j + 1] == sx[i]:
            j += 1
        ranks[order[i : j + 1]] = (i + j) / 2.0 + 1.0
        i = j + 1
    rank_sum_pos = ranks[y_true].sum()
    return float((rank_sum_pos - n_pos * (n_pos + 1) / 2.0) / (n_pos * n_neg))


def rmse_score(y_true, y_score) -> float:
    y_true = np.asarray(y_true, dtype=float)
```

```
    y_score = np.asarray(y_score, dtype=float)
    return float(np.sqrt(np.mean((y_score - y_true) ** 2)))


def generate_benchmark_data(
    n_concepts: int = 10,
    n_students: int = 150,
    seq_len: int = DEFAULT_SEQ_LEN,
    seed: int = 0,
) -> dict[int, np.ndarray]:
    """按概念生成 (n_students, seq_len) 的 0/1 矩阵。

    每个概念一套真实 BKT 参数（随机采样），每个学生叠加一个能力偏移
    （改变其 p_L0），制造跨学生异质性——这是预测任务有区分度的来源。
    """
    rng = np.random.default_rng(seed)
    data: dict[int, np.ndarray] = {}
    for c in range(n_concepts):
        p_T = rng.uniform(0.05, 0.25)
        p_S = rng.uniform(0.05, 0.20)
        p_G = rng.uniform(0.10, 0.35)
        p_L0_base = rng.uniform(0.20, 0.60)
        ability = rng.normal(0.0, 0.18, size=n_students)
        p_L0 = np.clip(p_L0_base + ability, 0.05, 0.90)

        obs = np.empty((n_students, seq_len), dtype=bool)
        learned = rng.random(n_students) < p_L0
        for t in range(seq_len):
            p_correct = np.where(learned, 1.0 - p_S, p_G)
            obs[:, t] = rng.random(n_students) < p_correct
            learned = learned | (rng.random(n_students) < p_T)
        data[c] = obs
    return data


def _recent_mean_prefix(seq: np.ndarray, t: int, k: int, fallback: float) -> float:
    """t 时刻之前最近 k 次作答的正确率；无历史时用全局均值兜底。"""
    history = np.asarray(seq[max(0, t - k) : t], dtype=float)
    return float(history.mean()) if history.size else fallback


def run_benchmark(
    n_concepts: int = 10,
    n_students: int = 150,
    seq_len: int = DEFAULT_SEQ_LEN,
    train_len: int = DEFAULT_TRAIN_LEN,
    seed: int = 0,
) -> list[dict[str, float | str]]:
    """返回 benchmark 表行：model × AUC × RMSE。"""
    if not 0 < train_len < seq_len:
```

```
851         raise ValueError("need 0 < train_len < seq_len")
852     data = generate_benchmark_data(n_concepts, n_students, seq_len, seed)
853 
854     # 每个概念只用训练段拟合参数
855     fitted: dict[int, BKTParams] = {}
856     global_train_rate = float(
857         np.mean([obs[:, :train_len].mean() for obs in data.values()])
858     )
859     for c, obs in data.items():
860         fitted[c] = fit(list(obs[:, :train_len]))
861 
862     y_true: list[float] = []
863     y_bkt: list[float] = []
864     y_global: list[float] = []
865     y_recent: list[float] = []
866     for c, obs in data.items():
867         params = fitted[c]
868         for seq in obs:
869             for t in range(train_len, seq_len):
870                 y_true.append(float(seq[t]))
871                 y_bkt.append(predict_next(seq[:t], params))
872                 y_global.append(global_train_rate)
873                 y_recent.append(_recent_mean_prefix(seq, t, k=3, fallback=global_train_rate))
874 
875     rows = []
876     for name, scores in (
877         ("bkt-ours (EM fit)", y_bkt),
878         ("recent-3 mean", y_recent),
879         ("global mean", y_global),
880     ):
881         rows.append(
882             {
883                 "model": name,
884                 "auc": round(auc_score(y_true, scores), 4),
885                 "rmse": round(rmse_score(y_true, scores), 4),
886             }
887         )
888     return rows
889 
890 
891 def format_markdown(rows: list[dict[str, float | str]]) -> str:
892     lines = [
893         "# benchmark — 下次作答预测（合成数据基线）",
894         "",
895         "- 数据：`examdiag.eval.generate_benchmark_data` 合成序列，10 概念 × 150 学生 × 12 步，"
896         "前 8 步训练、后 4 步在线预测（每步观测前给出 P(答对)），seed=0 可复现。",
897         "- 指标：AUC（带并列秩的 Mann-Whitney 实现）、RMSE。",
898         "- 数据集说明：按 SPEC 5.4 允许的路径使用仓库内自造合成序列；ASSISTments 真实数据"
899         "下载与 pyBKT 交叉比对留待后续 Wave。",
900         "",
```

```
        "| model | AUC | RMSE |",
        "|---|---|---|",
    ]
    for r in rows:
        lines.append(f"| {r['model']} | {r['auc']} | {r['rmse']} |")
    lines.append("")
    return "\n".join(lines)


def main() -> None:
    rows = run_benchmark()
    text = format_markdown(rows)
    out = Path(__file__).resolve().parent.parent / "benchmark.md"
    out.write_text(text, encoding="utf-8")
    print(text)
    print(f"written to {out}")


if __name__ == "__main__":
    main()

# ============ 文件: examdiag/cli.py  (129 行) ============
"""Command-line interface.

    examdiag diagnose --attempts data/demo_attempts.jsonl --concepts data/concepts/
        [--questions data/questions/] [--user ID] [--top 3] [--plan] [--json]

--concepts / --questions 都接受单文件或目录 (目录时合并全部 *.yaml / *.jsonl) 。
--questions 缺省时自动取 --concepts 的同级 questions 目录 (data/concepts/ -> data/questions/) 。
"""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

from rich.console import Console
from rich.table import Table

from . import __version__
from .diagnose import Diagnosis, diagnose
from .schema import ConceptGraph, load_attempts, load_concepts, load_questions
from .srs import plan_week


def _expand_concepts(path: Path) -> ConceptGraph:
    files = sorted(path.glob("*.yaml")) if path.is_dir() else [path]
    if not files:
```

```
        raise FileNotFoundError(f"no concepts yaml under {path}")
    merged: dict[str, object] = {}
    for f in files:
        merged.update(load_concepts(f).concepts)
    return ConceptGraph(concepts=merged)  # type: ignore[arg-type]


def _expand_questions(path: Path | None, concepts_path: Path) -> list:
    if path is None:
        parent = concepts_path.parent if concepts_path.is_file() else concepts_path
        path = parent.parent / "questions"
    files = sorted(path.glob("*.jsonl")) if path.is_dir() else [path]
    if not files:
        raise FileNotFoundError(f"no questions jsonl under {path}")
    out = []
    for f in files:
        out.extend(load_questions(f))
    return out


def _render(diag: Diagnosis, concepts: ConceptGraph, console: Console, show_plan: bool) -> None:
    if not diag.root_causes:
        console.print("[green]没有发现需要归因的错误模式（无错题或样本不足）。[/green]")
    table = Table(title="根因诊断报告", title_style="bold", header_style="bold cyan")
    table.add_column("#", justify="right")
    table.add_column("根因概念")
    table.add_column("掌握度", justify="right")
    table.add_column("涉及错题", justify="right")
    table.add_column("补习路径（按序补）")
    for i, r in enumerate(diag.root_causes, 1):
        mastery_color = "red" if r.mastery < 0.3 else "yellow"
        path_names = " → ".join(
            f"{concepts.concepts[cid].name}({cid})" for cid in r.remedy_path
        )
        table.add_row(
            str(i),
            f"[bold]{r.name}[/bold] [dim]{r.concept_id}[/dim]",
            f"[{mastery_color}]{r.mastery:.2f}[/{mastery_color}]",
            str(r.wrong_count),
            path_names,
        )
    console.print(table)
    if diag.insufficient_data:
        ids = ", ".join(diag.insufficient_data)
        console.print(f"[dim]insufficient_data（作答 < 3 次，不下结论）：{ids}[/dim]")
    if show_plan and diag.root_causes:
        plan_table = Table(title="未来 7 天复习计划", header_style="bold magenta")
        plan_table.add_column("天", justify="right")
        plan_table.add_column("复习概念（最弱优先）")
        names = {cid: f"{c.name}({cid})" for cid, c in concepts.concepts.items()}
```

```
        for p in plan_week(diag):
            plan_table.add_row(
                f"Day {p.day}",
                ", ".join(names[cid] for cid in p.concepts) or "[dim]休息[/dim]",
            )
        console.print(plan_table)


def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(
        prog="examdiag",
        description="Find the root-cause concept behind every wrong answer.",
    )
    parser.add_argument("--version", action="version", version=f"examdiag {__version__}")
    sub = parser.add_subparsers(dest="command", required=True)

    d = sub.add_parser("diagnose", help="诊断错题的根因概念")
    d.add_argument("--attempts", required=True, help="作答记录 jsonl（单文件）")
    d.add_argument("--concepts", required=True, help="概念图谱 yaml 文件或目录")
    d.add_argument("--questions", default=None, help="题库 jsonl 文件或目录（缺省自动发现）")
    d.add_argument("--user", default=None, help="只诊断该用户（缺省合并全部）")
    d.add_argument("--top", type=int, default=3, help="最多输出几个根因")
    d.add_argument("--plan", action="store_true", help="附带输出 7 天复习计划")
    d.add_argument("--json", action="store_true", help="以 JSON 输出（供脚本消费）")

    args = parser.parse_args(argv)
    console = Console()

    try:
        concepts = _expand_concepts(Path(args.concepts))
        questions = _expand_questions(
            Path(args.questions) if args.questions else None, Path(args.concepts)
        )
        attempts = load_attempts(args.attempts)
        diag = diagnose(attempts, questions, concepts, user_id=args.user, top_n=args.top)
    except (FileNotFoundError, KeyError, ValueError) as e:
        console.print(f"[red]error:[/red] {e}")
        return 1

    if args.json:
        payload = diag.to_dict()
        if args.plan:
            payload["plan"] = [p.to_dict() for p in plan_week(diag)]
        print(json.dumps(payload, ensure_ascii=False, indent=2))
    else:
        _render(diag, concepts, console, show_plan=args.plan)
    return 0


if __name__ == "__main__":
```

```
    sys.exit(main())

# ============ 文件: /Users/xi/Documents/exam-diag/tests/test_bkt.py (125 行) ============
import numpy as np
import pytest

from examdiag.bkt import (
    BKTParams,
    _group_stats,
    fit,
    forward_posterior,
    log_likelihood,
    predict_next,
    simulate,
)

TRUE = BKTParams(p_L0=0.3, p_T=0.1, p_S=0.1, p_G=0.2)


# ---------- 前向后验 ----------


def test_all_correct_raises_posterior_toward_high_value():
    post = forward_posterior([1, 1, 1, 1, 1], TRUE)
    assert post[0] > TRUE.p_L0
    assert np.all(np.diff(post) > 0)
    assert post[-1] > 0.85


def test_wrong_answer_drops_posterior():
    post = forward_posterior([1, 1, 0], TRUE)
    assert post[2] < post[1]
    # 答错但仍高于未作答时的先验：slip 有限
    assert post[2] > 0.3


def test_posterior_stays_in_unit_interval_on_long_sequence():
    rng = np.random.default_rng(7)
    seq = rng.integers(0, 2, size=200)
    post = forward_posterior(seq, TRUE)
    assert np.all((post >= 0) & (post <= 1))


def test_predict_next_between_emission_bounds():
    seq = [1, 1, 1, 0, 1]
    pred = predict_next(seq, TRUE)
    assert TRUE.p_G - 1e-9 <= pred <= 1 - TRUE.p_S + 1e-9
    # 连续答对后的预测应高于初始正确率
    assert predict_next([1] * 5, TRUE) > (1 - TRUE.p_S) * 0.5 + TRUE.p_G * 0.5

```

```
1101
1102  # ---------- EM 充分统计量 ----------
1103
1104
1105  def _reference_fb(seq, p: BKTParams):
1106      """无缩放的标量前向-后向参照实现，用于交叉验证批量实现。"""
1107      T = len(seq)
1108      b = [(1 - p.p_S, p.p_G) if x else (p.p_S, 1 - p.p_G) for x in seq]  # (P(o|L), P(o|U))
1109      alpha = np.empty((T, 2))
1110      alpha[0] = [p.p_L0 * b[0][0], (1 - p.p_L0) * b[0][1]]
1111      for t in range(1, T):
1112          alpha[t, 0] = (alpha[t - 1, 0] + p.p_T * alpha[t - 1, 1]) * b[t][0]
1113          alpha[t, 1] = (1 - p.p_T) * alpha[t - 1, 1] * b[t][1]
1114      ll = alpha[-1].sum()
1115      beta = np.ones((T, 2))
1116      for t in range(T - 2, -1, -1):
1117          beta[t, 0] = b[t + 1][0] * beta[t + 1, 0]
1118          beta[t, 1] = (
1119              p.p_T * b[t + 1][0] * beta[t + 1, 0] + (1 - p.p_T) * b[t + 1][1] * beta[t + 1, 1]
1120          )
1121      gamma = alpha * beta / ll  # 平滑后验 P(state_t | 整条序列)
1122      return float(np.log(ll)), gamma
1123
1124
1125  def test_group_stats_matches_reference_forward_backward():
1126      obs = np.array([[1, 1, 0, 1], [0, 1, 1, 1]], dtype=bool)
1127      stats = _group_stats(obs, TRUE)
1128      ll_ref = sum(_reference_fb(seq, TRUE)[0] for seq in obs)
1129      g0_ref = sum(_reference_fb(seq, TRUE)[1][0, 0] for seq in obs)
1130      assert stats["loglik"] == pytest.approx(ll_ref, rel=1e-9)
1131      # EM 用的是平滑后验，所以 gamma_L0 不等于过滤后验之和
1132      assert stats["gamma_L0"] == pytest.approx(g0_ref, rel=1e-9)
1133      # 每步 gamma_L + gamma_U = 1，聚合即 N_L + N_U = n*T
1134      assert stats["N_L"] + stats["N_U"] == pytest.approx(obs.size)
1135
1136
1137  def test_log_likelihood_improves_with_fitting():
1138      obs = simulate(TRUE, n_students=200, seq_len=20, seed=3)
1139      seqs = list(obs)
1140      wrong = BKTParams(p_L0=0.9, p_T=0.01, p_S=0.4, p_G=0.01)
1141      ll_before = log_likelihood(seqs, wrong)
1142      fitted = fit(seqs, init=wrong)
1143      ll_after = log_likelihood(seqs, fitted)
1144      assert ll_after > ll_before
1145
1146
1147  # ---------- 参数恢复 (Wave 0 验收硬指标) ----------
1148
1149
1150  def test_em_recovers_true_params():
```

```
    """真实参数生成 2000 条长度 30 的序列，EM 拟合后每个参数误差 < 0.05。"""
    seqs = list(simulate(TRUE, n_students=2000, seq_len=30, seed=42))
    fitted = fit(seqs, init=BKTParams(p_L0=0.4, p_T=0.15, p_S=0.15, p_G=0.15))
    errs = np.abs(fitted.as_array() - TRUE.as_array())
    for name, err in zip(("p_L0", "p_T", "p_S", "p_G"), errs, strict=True):
        assert err < 0.05, f"{name}: fitted={getattr(fitted, name):.4f} err={err:.4f}"


def test_em_recovers_params_from_variable_length_sequences():
    rng = np.random.default_rng(11)
    seqs = list(simulate(TRUE, n_students=1200, seq_len=30, seed=1))
    # 随机截断成长度 5..30 的变长序列
    short = [seq[: rng.integers(5, 31)] for seq in seqs[:800]]
    fitted = fit(short, init=BKTParams(p_L0=0.4, p_T=0.2, p_S=0.2, p_G=0.2))
    assert np.abs(fitted.as_array() - TRUE.as_array()).max() < 0.05


def test_fit_respects_param_bounds():
    # 极端初始化也不允许参数跑出 [0.01, 0.99]
    seqs = list(simulate(TRUE, n_students=100, seq_len=10, seed=5))
    fitted = fit(seqs, init=BKTParams(p_L0=0.99, p_T=0.99, p_S=0.01, p_G=0.01))
    assert np.all(fitted.as_array() >= 0.01 - 1e-12)
    assert np.all(fitted.as_array() <= 0.99 + 1e-12)


def test_fit_rejects_empty_input():
    with pytest.raises(ValueError):
        fit([])

# ============ 文件: /Users/xi/Documents/exam-diag/tests/test_cli.py  (113 行) ============
import json
from datetime import datetime, timedelta

import pytest

from examdiag.cli import _expand_concepts, _expand_questions, main
from examdiag.schema import Attempt, Question

T0 = datetime(2026, 10, 20, 9, 0, 0)


@pytest.fixture
def world(tmp_path):
    """data 布局复刻：concepts/ 两科 yaml + questions/ 两科 jsonl + attempts.jsonl。"""
    cdir = tmp_path / "concepts"
    qdir = tmp_path / "questions"
    cdir.mkdir()
    qdir.mkdir()

    (cdir / "s1.yaml").write_text(
```

```
        "- {id: s.a, name: A基础, subject: 408, difficulty: 2, prereqs: []}\n"
        "- {id: s.b, name: B中级, subject: 408, difficulty: 3, prereqs: [s.a]}\n",
        encoding="utf-8",
    )
    (cdir / "s2.yaml").write_text(
        "- {id: s.d, name: D独立, subject: 408, difficulty: 3, prereqs: []}\n",
        encoding="utf-8",
    )
    questions = [
        *[Question(id=f"q-a{i}", subject="408", stem="s", answer="A",
                   concepts=["s.a"], source="self-built") for i in range(4)],
        *[Question(id=f"q-b{i}", subject="408", stem="s", answer="A",
                   concepts=["s.b"], source="self-built") for i in range(4)],
        *[Question(id=f"q-d{i}", subject="408", stem="s", answer="A",
                   concepts=["s.d"], source="self-built") for i in range(4)],
    ]
    with open(qdir / "s1.jsonl", "w", encoding="utf-8") as f:
        for q in questions[:8]:
            f.write(q.model_dump_json() + "\n")
    with open(qdir / "s2.jsonl", "w", encoding="utf-8") as f:
        for q in questions[8:]:
            f.write(q.model_dump_json() + "\n")

    attempts, t = [], T0
    for qid, ok in [("q-a0", 1), ("q-a1", 1), ("q-a2", 1), ("q-a3", 1),
                    ("q-b0", 0), ("q-b1", 0), ("q-b2", 1), ("q-b3", 0),
                    ("q-d0", 0)]:
        attempts.append(Attempt(user_id="u1", question_id=qid, correct=bool(ok), ts=t))
        t += timedelta(minutes=5)
    with open(tmp_path / "attempts.jsonl", "w", encoding="utf-8") as f:
        for a in attempts:
            f.write(a.model_dump_json() + "\n")
    return tmp_path


def test_expand_concepts_merges_directory(world):
    g = _expand_concepts(world / "concepts")
    assert set(g.concepts) == {"s.a", "s.b", "s.d"}


def test_expand_questions_directory_and_default_discovery(world):
    qs = _expand_questions(world / "questions", world / "concepts")
    assert len(qs) == 12
    # 缺省 = concepts 同级的 questions 目录
    assert len(_expand_questions(None, world / "concepts")) == 12


def test_cli_json_first_root_cause(world, capsys):
    rc = main([
        "diagnose",
```

```
        "--attempts", str(world / "attempts.jsonl"),
        "--concepts", str(world / "concepts"),
        "--json",
    ])
    assert rc == 0
    payload = json.loads(capsys.readouterr().out)
    assert payload["root_causes"][0]["concept_id"] == "s.b"
    assert "s.d" in payload["insufficient_data"]  # 只答 1 次


def test_cli_table_render(world, capsys):
    rc = main([
        "diagnose",
        "--attempts", str(world / "attempts.jsonl"),
        "--concepts", str(world / "concepts"),
        "--plan",
    ])
    assert rc == 0
    out = capsys.readouterr().out
    assert "根因诊断报告" in out
    assert "B中级" in out
    assert "未来 7 天复习计划" in out


def test_cli_missing_file_returns_error(world, capsys):
    rc = main([
        "diagnose",
        "--attempts", str(world / "nope.jsonl"),
        "--concepts", str(world / "concepts"),
    ])
    assert rc == 1
    assert "error" in capsys.readouterr().out


def test_cli_user_filter_and_top(world, capsys):
    rc = main([
        "diagnose",
        "--attempts", str(world / "attempts.jsonl"),
        "--concepts", str(world / "concepts"),
        "--user", "nobody",
        "--json",
    ])
    assert rc == 1  # 该用户无作答 → 报错退出

# ============ 文件: /Users/xi/Documents/exam-diag/tests/test_demo_acceptance.py (57 行) ============
"""Wave 1 验收：demo 数据上诊断出的第 1 个根因必须是 ds.tree.traverse (SPEC 1.6) 。"""

from pathlib import Path

import pytest
```

```
1301
1302 from examdiag.diagnose import diagnose
1303 from examdiag.schema import load_attempts, load_concepts, load_questions
1304
1305 ROOT = Path(__file__).resolve().parent.parent
1306 DEMO = ROOT / "data/demo_attempts.jsonl"
1307
1308 pytestmark = pytest.mark.skipif(
1309     not DEMO.exists(), reason="data/demo_attempts.jsonl not generated yet"
1310 )
1311
1312
1313 @pytest.fixture(scope="module")
1314 def demo_diagnosis():
1315     concepts = load_concepts(ROOT / "data/concepts/408.yaml")
1316     questions = load_questions(ROOT / "data/questions/408.jsonl")
1317     attempts = load_attempts(DEMO)
1318     return diagnose(attempts, questions, concepts), attempts
1319
1320
1321 def test_demo_first_root_cause_is_tree_traverse(demo_diagnosis):
1322     d, _ = demo_diagnosis
1323     assert d.root_causes, "demo 数据必须产出根因"
1324     assert d.root_causes[0].concept_id == "ds.tree.traverse"
1325
1326
1327 def test_demo_rotate_failures_explained_by_traverse(demo_diagnosis):
1328     """植入的『BST 旋转连续错』必须被归因为 traverse 的下游症状，而非独立根因。"""
1329     d, _ = demo_diagnosis
1330     ids = [r.concept_id for r in d.root_causes]
1331     assert "ds.bst.rotate" not in ids
1332     assert "ds.bst.rotate" in d.root_causes[0].explains
1333
1334
1335 def test_demo_remedy_path_goes_through_prereq_chain(demo_diagnosis):
1336     d, _ = demo_diagnosis
1337     path = d.root_causes[0].remedy_path
1338     assert path[0] == "ds.tree.traverse"
1339     assert "ds.bst.rotate" in path  # 补习路径覆盖到旋转
1340
1341
1342 def test_demo_prereqs_stay_mastered(demo_diagnosis):
1343     d, _ = demo_diagnosis
1344     for cid in ("ds.stack.stack", "ds.recursion", "ds.tree.binary"):
1345         assert d.mastery[cid] > 0.5, f"{cid} 应保持掌握（demo 植入为答对）"
1346     assert d.mastery["ds.tree.traverse"] < 0.5
1347
1348
1349 def test_demo_attempt_count(demo_diagnosis):
1350     _, attempts = demo_diagnosis
```

```
    assert len(attempts) == 500
    assert {a.user_id for a in attempts} == {"demo-user"}

# ============ 文件: /Users/xi/Documents/exam-diag/tests/test_diagnose.py (205 行) ============
from datetime import datetime, timedelta

import pytest

from examdiag.diagnose import (
    MASTERY_THRESHOLD,
    MIN_ATTEMPTS,
    Diagnosis,
    collect_sequences,
    diagnose,
    estimate_mastery,
    score,
)
from examdiag.schema import Attempt, Concept, ConceptGraph, Question

T0 = datetime(2026, 10, 20, 9, 0, 0)


def build_world():
    """链：a -> b -> c；旁支：d（无前置）、e（前置 d）。

    a=栈(会), b=遍历(不会), c=旋转(不会) 的最小复刻。
    """
    concepts = ConceptGraph.from_list([
        Concept(id="s.a", name="A基础", subject="408", difficulty=2, prereqs=[]),
        Concept(id="s.b", name="B中级", subject="408", difficulty=3, prereqs=["s.a"]),
        Concept(id="s.c", name="C高级", subject="408", difficulty=5, prereqs=["s.b"]),
        Concept(id="s.d", name="D独立", subject="408", difficulty=3, prereqs=[]),
        Concept(id="s.e", name="E依赖D", subject="408", difficulty=4, prereqs=["s.d"]),
    ])
    questions = []
    pairs = [("q-a", "s.a"), ("q-b", "s.b"), ("q-c", "s.c"), ("q-d", "s.d"), ("q-e", "s.e")]
    for prefix, cid in pairs:
        questions += [
            Question(id=f"{prefix}{i}", subject="408", stem="s", answer="A",
                     concepts=[cid], source="self-built")
            for i in range(4)
        ]
    return concepts, questions


def attempts_for(pattern: dict[str, list[int]]):
    """pattern: 问题 id 前缀 -> 作答序列 (0/1)，时间依次递增。"""
    out = []
    t = T0
    for prefix, answers in pattern.items():
```

```
        for i, ok in enumerate(answers):
            out.append(
                Attempt(user_id="u1", question_id=f"{prefix}{i}", correct=bool(ok), ts=t)
            )
            t += timedelta(minutes=10)
    return out


# ---------- collect_sequences / estimate_mastery ----------


def test_collect_sequences_sorts_by_ts_and_fans_out_concepts():
    concepts, questions = build_world()
    qmulti = Question(id="q-multi", subject="408", stem="s", answer="A",
                      concepts=["s.a", "s.b"], source="self-built")
    attempts = [
        Attempt(user_id="u1", question_id="q-multi", correct=True, ts=T0 + timedelta(minutes=5)),
        Attempt(user_id="u1", question_id="q-a0", correct=False, ts=T0),
    ]
    seqs = collect_sequences(attempts, [*questions, qmulti])
    assert seqs["s.a"] == [0, 1]  # 按 ts 排序后 a 先错后对
    assert seqs["s.b"] == [1]
    assert "s.c" not in seqs


def test_collect_sequences_unknown_question_raises():
    _, questions = build_world()
    ghost = Attempt(user_id="u1", question_id="ghost", correct=True, ts=T0)
    with pytest.raises(KeyError):
        collect_sequences([ghost], questions)


def test_estimate_mastery_marks_insufficient_below_min_attempts():
    seqs = {"s.a": [1, 1, 1], "s.b": [0, 0]}
    mastery, insufficient = estimate_mastery(seqs)
    assert "s.a" in mastery
    assert insufficient == ["s.b"]
    assert len(seqs) == 2 and MIN_ATTEMPTS == 3


def test_collect_sequences_user_filter():
    concepts, questions = build_world()
    attempts = [
        Attempt(user_id="u1", question_id="q-a0", correct=True, ts=T0),
        Attempt(user_id="u2", question_id="q-a1", correct=False, ts=T0),
    ]
    seqs = collect_sequences(attempts, questions, user_id="u2")
    assert seqs == {"s.a": [0]}
    with pytest.raises(KeyError):
        collect_sequences(attempts, questions, user_id="nobody")
```

```
1451
1452
1453  # ---------- score 公式（SPEC 5.3 字面实现） ----------
1454
1455
1456  def test_score_matches_spec_formula():
1457      g_concepts, _ = build_world()
1458      from examdiag.graph import build
1459
1460      g = build(g_concepts)
1461      # a(0.9) -> b(0.2) -> c(0.5)
1462      m = {"s.a": 0.9, "s.b": 0.2, "s.c": 0.5}
1463      assert score(g, "s.a", m) == pytest.approx(1 - 0.9)  # 无有数据前置
1464      assert score(g, "s.b", m) == pytest.approx(0.8 * 0.1)  # d=1
1465      assert score(g, "s.c", m) == pytest.approx(0.5 * 0.1 ** (1 / 2) * 0.8)  # b d=1, a d=2
1466
1467
1468  def test_score_skips_insufficient_ancestors():
1469      g_concepts, _ = build_world()
1470      from examdiag.graph import build
1471
1472      g = build(g_concepts)
1473      # s.a 无数据 → b 的乘积只剩它自己
1474      assert score(g, "s.b", {"s.b": 0.3}) == pytest.approx(0.7)
1475
1476
1477  # ---------- diagnose 主流程 ----------
1478
1479
1480  def test_symptom_filter_picks_upstream_root_cause():
1481      """a 掌握、b/c 连错 → 根因是 b（最上游未掌握），c 是症状被过滤。"""
1482      concepts, questions = build_world()
1483      attempts = attempts_for({
1484          "q-a": [1, 1, 1, 1],
1485          "q-b": [0, 0, 1, 0],
1486          "q-c": [0, 0, 0, 0],
1487          "q-d": [1, 1, 1, 1],
1488      })
1489      d = diagnose(attempts, questions, concepts)
1490      assert d.root_causes[0].concept_id == "s.b"
1491      ids = [r.concept_id for r in d.root_causes]
1492      assert "s.c" not in ids  # c 的前置 b 未掌握 → 症状
1493      assert d.root_causes[0].explains == ["s.c"]
1494      assert d.root_causes[0].remedy_path == ["s.b", "s.c"]
1495
1496
1497  def test_mastered_ancestor_keeps_downstream_as_root():
1498      """a 掌握、b 连错且 a 已会 → b 自己就是根因。"""
1499      concepts, questions = build_world()
1500      attempts = attempts_for({
```

```
        own_all += list(zip(res["own"]["preds"], res["own"]["truth"], strict=True))
        py_all += list(zip(res["py"]["preds"], res["py"]["truth"], strict=True))
        rows.append(f"| {res['fold']} | {res['n_train']}/{res['n_test']} | {o_a:.4f} | {o_r:.4f} | "
                    f"{p_a:.4f} | {p_r:.4f} | {res['fit_s']:.1f} |")
        print(f"fold {res['fold']}: own AUC={o_a:.4f} pyBKT AUC={p_a:.4f}", flush=True)

    own_a = auc_score([t for _, t in own_all], [p for p, _ in own_all])
    own_r = rmse_score([t for _, t in own_all], [p for p, _ in own_all])
    py_a = auc_score([t for _, t in py_all], [p for p, _ in py_all])
    py_r = rmse_score([t for _, t in py_all], [p for p, _ in py_all])
    gap = abs(own_a - py_a)

    verdict = "PASS" if gap < 0.02 else ("WARNING (差距在 0.02–0.05，继续) " if gap <= 0.05 else "STOP")
    rows += ["",
             f"- 汇总：自写 BKT **AUC {own_a:.4f} / RMSE {own_r:.4f}**；"
             f"pyBKT **AUC {py_a:.4f} / RMSE {py_r:.4f}**；AUC 差距 **{gap:.4f}**",
             f"- 预测对数：自写 {len(own_all):,}；pyBKT {len(py_all):,}",
             f"- 闸门 (SPEC 3.3) ：{'<0.02 通过' if gap < 0.02 else '>0.05 需 STOP' if gap > 0.05 else '0.02–0.05 未达通过线
    REPORT.write_text("\n".join(rows) + "\n", encoding="utf-8")
    print("\n".join(rows[-4:]))

    if gap > 0.05:
        (ROOT / "STOP").write_text(
            f"AUC gap {gap:.4f} > 0.05 on ASSISTments 5-fold CV\n"
            f"own={own_a:.4f} pyBKT={py_a:.4f}\nsee {REPORT}\n", encoding="utf-8")
        print("[STOP] AUC gap > 0.05 — STOP 文件已创建")
        raise SystemExit(2)


if __name__ == "__main__":
    main()

# ============ 文件: /Users/xi/Documents/exam-diag/scripts/convert_assistments.py (115 行) ============
# ruff: noqa: E501
"""把 ASSISTments skill-builder 原始 CSV 转成 examdiag 数据格式 (Wave 3 任务 3.2) 。

输入：data/real/raw/skill_builder_data.csv (官方 corrected 版，346,860 行)
输出：
  data/real/assistments.jsonl          Attempt 格式 {user_id, question_id, correct, ts}
  data/real/skills.jsonl               {question_id, skill} 唯一映射对
  data/real/concepts_assistments.yaml  101 个技能 → 概念节点 (与 Concept 字段同构)
  data/real/CONVERSION_REPORT.md       真实转换统计 (进/弃行数与原因)

规则 (全部如实统计，不许静默丢弃) ：
  - correct 必须 ∈ {0,1}，其余丢弃；
  - user_id / problem_id / skill_name 任一为空即丢弃；
  - ts 由 order_id 合成 (2000-01-01 起、秒级单调) ，BKT 只依赖序列顺序，
    原始数据无逐条时间戳，此为文档化处理而非编造；
  - skills 与 questions 一对一校验：同一 problem_id 若挂多个 skill，映射表按
    (question_id, skill) 唯一对输出并在报告计数。
```

```
1551
1552  运行：python scripts/convert_assistments.py
1553  """
1554
1555  from __future__ import annotations
1556
1557  import csv
1558  import json
1559  import re
1560  from collections import Counter, defaultdict
1561  from datetime import datetime, timedelta
1562  from pathlib import Path
1563
1564  ROOT = Path(__file__).resolve().parent.parent
1565  RAW = ROOT / "data/real/raw/skill_builder_data.csv"
1566  OUT_DIR = ROOT / "data/real"
1567  TS_BASE = datetime(2000, 1, 1)
1568
1569
1570  def slugify(name: str) -> str:
1571      s = name.strip().lower()
1572      s = re.sub(r"[^a-z0-9]+", "_", s).strip("_")
1573      return s or "skill"
1574
1575
1576  def main() -> None:
1577      stats: Counter[str] = Counter()
1578      rows_out: list[dict] = []
1579      q2skills: dict[str, set[str]] = defaultdict(set)
1580      skill_rows: Counter[str] = Counter()
1581
1582      with open(RAW, encoding="utf-8", errors="replace") as f:
1583          reader = csv.DictReader(f)
1584          for row in reader:
1585              stats["rows_in"] += 1
1586              uid = row["user_id"].strip()
1587              pid = row["problem_id"].strip()
1588              correct = row["correct"].strip()
1589              skill = row["skill_name"].strip()
1590              order = row["order_id"].strip()
1591              if not uid or not pid:
1592                  stats["drop_missing_user_or_problem"] += 1
1593                  continue
1594              if not skill:
1595                  stats["drop_empty_skill"] += 1
1596                  continue
1597              if correct not in ("0", "1"):
1598                  stats["drop_correct_not_binary"] += 1
1599                  continue
1600              qid = f"p{pid}"
```

```
            ts = (TS_BASE + timedelta(seconds=int(order))).isoformat()
            rows_out.append(
                {"user_id": uid, "question_id": qid, "correct": correct == "1", "ts": ts}
            )
            q2skills[qid].add(skill)
            skill_rows[skill] += 1
            stats["attempts_out"] += 1

    OUT_DIR.mkdir(parents=True, exist_ok=True)
    with open(OUT_DIR / "assistments.jsonl", "w", encoding="utf-8") as f:
        for r in rows_out:
            f.write(json.dumps(r, ensure_ascii=False) + "\n")
    with open(OUT_DIR / "skills.jsonl", "w", encoding="utf-8") as f:
        for qid in sorted(q2skills):
            for sk in sorted(q2skills[qid]):
                f.write(json.dumps({"question_id": qid, "skill": sk}, ensure_ascii=False) + "\n")

    # 概念文件（与 Concept 字段同构；subject 为数据集名，
    # 不走 examdiag.schema 四科枚举校验—真实数据集不属于四科）
    lines = ["# ASSISTments skill-builder 概念（= 数据集自带 skill 标注）",
             "# difficulty 真实值未知，统一中性 3.0；prereqs 空（BKT 逐技能独立，无前置图）",
             "subject: \"assistments\"", "concepts:"]
    for sk in sorted(skill_rows):
        lines.append(
            f"  - {{id: asst.{slugify(sk)}, name: \"{sk}\", subject: \"assistments\", "
            f"difficulty: 3.0, prereqs: []}}"
        )
    (OUT_DIR / "concepts_assistments.yaml").write_text("\n".join(lines) + "\n", encoding="utf-8")

    users = {r["user_id"] for r in rows_out}
    multi = sum(1 for q, s in q2skills.items() if len(s) > 1)
    report = [
        "# ASSISTments 转换报告（真实统计）", "",
        f"- 输入行数：{stats['rows_in']:,}",
        f"- 输出 attempts：{stats['attempts_out']:,}（用户 {len(users):,}，question "
        f"{len(q2skills):,}，skill {len(skill_rows)}）",
        f"- 丢弃行：缺 user/problem {stats['drop_missing_user_or_problem']:,}；"
        f"空 skill {stats['drop_empty_skill']:,}；correct 非 0/1 "
        f"{stats['drop_correct_not_binary']:,}",
        f"- 一题多技能：{multi} 个 question（映射表按 (question_id, skill) 唯一对展开）",
        "- ts 说明：由 order_id 合成单调时间戳（原始数据无逐条时间）",
    ]
    (OUT_DIR / "CONVERSION_REPORT.md").write_text("\n".join(report) + "\n", encoding="utf-8")
    print("\n".join(report))


if __name__ == "__main__":
    main()

# ============ 文件: /Users/xi/Documents/exam-diag/scripts/download_assistments.py （111 行） ============
```

```
# ruff: noqa: E501
"""下载 ASSISTments skill-builder 数据集（Wave 3 任务 3.1）。

策略：按候选 URL 列表依次尝试，每个请求限时（连接/下载双超时），全部失败则
写 DOWNLOAD_FAILED.md（含每次尝试的真实原因），绝不无限等待。

候选源（按优先级）：
  1-2. 官方 Google Sites 原始文件（skill_builder_data_corrected.csv / .csv）
  3.   hf-mirror.com 镜像 OloriBern/assistments-dataset（官方 corrected 版，64.4MB）
       —— Google Sites 在本机网络不可达时的实际落点
  4.   GitHub codeload 镜像 psp0/Metaverse_Edu_Platform（sakt/data/skill_builder_data.csv）

下载成功后校验：非空、以表头行开头、包含关键列（user_id/correct/skill）。
输出：data/real/raw/skill_builder_data.csv
运行：python scripts/download_assistments.py
"""

from __future__ import annotations

import shutil
import subprocess
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
DEST = ROOT / "data/real/raw/skill_builder_data.csv"
HF_BASE = "https://hf-mirror.com/datasets/OloriBern/assistments-dataset/resolve/main"

CANDIDATES = [
    ("official-google-sites (corrected)",
     "https://sites.google.com/site/assistmentsdata/home/2009-2010-assistment-data/"
     "skill_builder_data_corrected.csv?attredirects=0"),
    ("official-google-sites (original)",
     "https://sites.google.com/site/assistmentsdata/home/2009-2010-assistment-data/"
     "skill_builder_data.csv?attredirects=0"),
    ("hf-mirror OloriBern/assistments-dataset (corrected)",
     f"{HF_BASE}/skill_builder_data_corrected.csv"),
    ("github-codeload psp0/Metaverse_Edu_Platform",
     "https://codeload.github.com/psp0/Metaverse_Edu_Platform/zip/refs/heads/main"),
]

REQUIRED_COLS = ("user_id", "correct", "skill")


def try_download(url: str, workdir: Path) -> tuple[Path | None, str]:
    """单候选下载。返回（文件路径 | None，结果说明）。限时 60s 连接 / 600s 总量。"""
    target = workdir / ("candidate_" + url.split("/")[-1].split("?")[0])
    cmd = [
        "curl", "-L", "--fail", "--silent", "--show-error",
        "--connect-timeout", "60", "--max-time", "600",
        "-o", str(target), url,
```

```
    ]
    try:
        proc = subprocess.run(cmd, capture_output=True, text=True, timeout=660)
    except subprocess.TimeoutExpired:
        return None, "timeout after 660s (killed)"
    if proc.returncode != 0:
        return None, f"curl rc={proc.returncode}: {proc.stderr.strip()[:300] or 'no stderr (blocked/empty response)'}"
    if not target.exists() or target.stat().st_size == 0:
        return None, "empty file"
    return target, f"ok ({target.stat().st_size / 1024 / 1024:.1f} MB)"


def validate(path: Path) -> tuple[bool, str]:
    head = path.read_text(encoding="utf-8", errors="replace").split("\n", 1)[0].lower()
    missing = [c for c in REQUIRED_COLS if c not in head]
    if missing:
        return False, f"header check failed, missing columns {missing}; header={head[:160]}"
    return True, f"header ok ({head[:120]}...)"


def main() -> int:
    if DEST.exists() and DEST.stat().st_size > 1_000_000:
        print(f"already downloaded: {DEST} ({DEST.stat().st_size / 1024 / 1024:.1f} MB)")
        return 0
    DEST.parent.mkdir(parents=True, exist_ok=True)
    workdir = DEST.parent
    log: list[str] = ["# ASSISTments 下载报告", ""]

    for name, url in CANDIDATES:
        print(f"try: {name} ...", flush=True)
        got, msg = try_download(url, workdir)
        log.append(f"- **{name}** → {msg}")
        print(f"    {msg}", flush=True)
        if got is None:
            continue
        if name.startswith("github-codeload"):
            # zip 包：解开找 csv
            shutil.unpack_archive(got, workdir / "zip_extract")
            found = sorted((workdir / "zip_extract").rglob("skill_builder_data*.csv"))
            if not found:
                log.append("  - zip 内未找到 skill_builder_data*.csv")
                continue
            got = found[0]
        ok, why = validate(got)
        log.append(f"  - 校验: {why}")
        print(f"    validate: {why}", flush=True)
        if ok:
            shutil.copyfile(got, DEST)
            log.append(f"\n落盘：`{DEST.relative_to(ROOT)}` ({DEST.stat().st_size / 1024 / 1024:.1f} MB) ")
            (workdir / "DOWNLOAD_LOG.md").write_text("\n".join(log) + "\n", encoding="utf-8")
```

```
            print(f"saved {DEST}")
            return 0

    log.append("\n全部候选失败（原因见上）。")
    (ROOT / "DOWNLOAD_FAILED.md").write_text("\n".join(log) + "\n", encoding="utf-8")
    print("DOWNLOAD_FAILED.md written")
    return 1


if __name__ == "__main__":
    raise SystemExit(main())

# ============ 文件: /Users/xi/Documents/exam-diag/scripts/extract_source.py （135 行） ============
# ruff: noqa: E501
"""软著申请材料 4.1：源程序 PDF（docs/copyright/source-60p.pdf）。

内容：仓库全部真实 Python 源码（examdiag/ 核心包 1,036 行 + tests/ 897 行 +
scripts/ 1,318 行，共约 3,251 行；examdiag/ 单独不足 30 页，故按"全程序"收录，
构成在页眉下方与报告中如实注明）。
排版：每页 50 行、行号、页眉含软件全称 + 版本号 + 页码；
    总页数 > 60 时按软著惯例取前 30 页 + 后 30 页。
渲染：PIL 逐页绘制（Menlo + STHeiti 逐字符回退），PIL 多页合成 A4 PDF（150dpi）。
页数用 pypdf 实读校验。

运行：python scripts/extract_source.py
"""

from __future__ import annotations

import sys
from pathlib import Path

from PIL import Image, ImageDraw, ImageFont
from pypdf import PdfReader

ROOT = Path(__file__).resolve().parent.parent
OUT = ROOT / "docs/copyright/source-60p.pdf"

SOFTWARE = "exam-diag 考试错题根因诊断系统"
VERSION = "V0.5.0"
LINES_PER_PAGE = 50
DPI = 150
PAGE_W, PAGE_H = 1240, 1754  # A4 @150dpi
MARGIN_X, MARGIN_Y = 50, 56
ROW_H = 30
SIZE = 15  # Menlo
SIZE_CJK = 19  # STHeiti (≈2 格宽)

FONT_LATIN = "/System/Library/Fonts/Menlo.ttc"
FONT_CJK = "/System/Library/Fonts/STHeiti Medium.ttc"
```

```
1801
1802 FILE_ORDER = [
1803     "examdiag/__init__.py", "examdiag/schema.py", "examdiag/bkt.py",
1804     "examdiag/graph.py", "examdiag/diagnose.py", "examdiag/srs.py",
1805     "examdiag/eval.py", "examdiag/cli.py",
1806 ]
1807
1808
1809 def load_fonts():
1810     return (
1811         ImageFont.truetype(FONT_LATIN, SIZE),
1812         ImageFont.truetype(FONT_CJK, SIZE_CJK),
1813     )
1814
1815
1816 def draw_mixed(draw, xy, text, fonts, fill):
1817     """逐字符混排（CJK 用 STHeiti），返回结束 x。"""
1818     fl, fc = fonts
1819     x, y = xy
1820     for ch in text:
1821         f = fc if ord(ch) >= 0x2E80 else fl
1822         draw.text((x, y), ch, font=f, fill=fill)
1823         x += f.getlength(ch)
1824     return x
1825
1826
1827 def collect_lines() -> tuple[list[str], dict[str, int]]:
1828     """按逻辑顺序收集全部源码行（文件头注释行计入，真实可审计）。"""
1829     lines: list[str] = []
1830     per_file: dict[str, int] = {}
1831     ordered = FILE_ORDER[:]
1832     ordered += sorted(p.as_posix() for p in Path(ROOT, "tests").glob("*.py"))
1833     ordered += sorted(p.as_posix() for p in Path(ROOT, "scripts").glob("*.py"))
1834     for rel in ordered:
1835         path = ROOT / rel
1836         if not path.exists():
1837             continue
1838         text = path.read_text(encoding="utf-8").rstrip("\n")
1839         body = text.splitlines()
1840         header = f"# ============ 文件: {rel}  ({len(body)} 行) ============"
1841         chunk = [header, *body, ""]
1842         per_file[rel] = len(body)
1843         lines.extend(chunk)
1844     return lines, per_file
1845
1846
1847 def paginate(lines: list[str]) -> list[list[str]]:
1848     pages = [lines[i : i + LINES_PER_PAGE] for i in range(0, len(lines), LINES_PER_PAGE)]
1849     if len(pages) > 60:  # 软著惯例：全程序超 60 页取前 30 + 后 30
1850         pages = pages[:30] + pages[-30:]
```

```
    return pages


def render_pages(pages: list[list[str]], fonts) -> list[Image.Image]:
    imgs = []
    total = len(pages)
    for idx, page_lines in enumerate(pages, 1):
        img = Image.new("RGB", (PAGE_W, PAGE_H), "white")
        d = ImageDraw.Draw(img)
        # 页眉：软件全称 + 版本号 + 页码
        draw_mixed(d, (MARGIN_X, 20), f"{SOFTWARE}  {VERSION}", fonts, "black")
        page_tag = f"源程序  第 {idx} 页 / 共 {total} 页"
        draw_mixed(d, (PAGE_W - MARGIN_X - 330, 20), page_tag, fonts, "black")
        d.line([(MARGIN_X, 52), (PAGE_W - MARGIN_X, 52)], fill="black", width=2)
        if idx == 1:
            note = "（收录范围：examdiag/ 核心包 + tests/ 测试 + scripts/ 工具脚本，全部真实源码）"
            draw_mixed(d, (MARGIN_X, 56), note, fonts, (90, 90, 90))
        y = 92 if idx == 1 else 72
        for i, line in enumerate(page_lines, 1):
            no = f"{(idx - 1) * LINES_PER_PAGE + i:>4}  "
            d.text((MARGIN_X, y), no, font=fonts[0], fill=(120, 120, 120))
            draw_mixed(d, (MARGIN_X + 58, y), line[:150], fonts, "black")
            y += ROW_H
        imgs.append(img)
    return imgs


def main() -> None:
    fonts = load_fonts()
    lines, per_file = collect_lines()
    total_lines = len(lines)
    pages = paginate(lines)
    imgs = render_pages(pages, fonts)

    OUT.parent.mkdir(parents=True, exist_ok=True)
    imgs[0].save(
        OUT, save_all=True, append_images=imgs[1:], resolution=DPI,
        title=f"{SOFTWARE} {VERSION} 源程序",
    )
    real_pages = len(PdfReader(OUT).pages)
    print(f"written {OUT}")
    print(f"source files: {len(per_file)}, listing lines: {total_lines:,}")
    print(f"examdiag/ core: {sum(v for k, v in per_file.items() if k.startswith('examdiag/')):,} lines")
    print(f"pages rendered: {len(pages)} (per-page {LINES_PER_PAGE} lines), PDF pages (pypdf): {real_pages}")


if __name__ == "__main__":
    sys.exit(main())

# ============ 文件: /Users/xi/Documents/exam-diag/scripts/make_demo_attempts.py （135 行） ============
```

```
"""生成 data/demo_attempts.jsonl：500 条单用户作答，植入可诊断的根因模式。

模式（SPEC Wave 1 任务 1.5）：
  - 结尾连续 10 次作答：ds.bst.rotate 全错 ×6 + ds.bst.delete 全错 ×4（"BST 旋转变形题连续错"）
  - ds.tree.traverse 先对 3 后连错 5（其前置"二叉树遍历"也错，且是下滑趋势）
  - 前置概念掌握良好：ds.stack.stack / ds.recursion 全对，ds.tree.binary 4/5，
    ds.bst.bst 3/5（基础 BST 尚可，旋转不行）
  - 其余 408 概念填充 ~464 条、80% 正确率（真实感背景）

时间线：按"多流归并"交错（保持每个概念内部的作答顺序），末尾追加连续错窗口。
seed 固定，输出可复现。运行：python scripts/make_demo_attempts.py
"""

from __future__ import annotations

import json
import random
from collections import defaultdict
from datetime import datetime, timedelta
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
SEED = 20261028
N_TOTAL = 500

# 植入模式：概念 -> 作答序列（1=对 0=错），顺序即该概念的时间顺序
PLANTED: dict[str, list[int]] = {
    "ds.stack.stack": [1, 1, 1, 1],
    "ds.recursion": [1, 1, 1, 1],
    "ds.tree.binary": [1, 1, 1, 1, 0],
    "ds.bst.bst": [1, 0, 1, 1, 0],
    "ds.tree.traverse": [1, 1, 1, 0, 0, 0, 0, 0],
}
ENDING_STREAK: dict[str, list[int]] = {
    "ds.bst.rotate": [0, 0, 0, 0, 0, 0],
    "ds.bst.delete": [0, 0, 0, 0],
}
FILLER_CORRECT_P = 0.80


def main() -> None:
    rng = random.Random(SEED)
    questions: dict[str, list] = defaultdict(list)
    all_q = []
    with open(ROOT / "data/questions/408.jsonl", encoding="utf-8") as f:
        for line in f:
            if line.strip():
                q = json.loads(line)
                all_q.append(q)
                for cid in q["concepts"]:
```

```
                questions[cid].append(q)

    planted_ids = set(PLANTED) | set(ENDING_STREAK)
    # 填充题池：主概念未植入、且整题不触碰任何植入概念（避免稀释植入模式）
    clean_questions: dict[str, list] = {}
    for cid, qs in questions.items():
        if cid in planted_ids:
            continue
        pool = [q for q in qs if not (set(q["concepts"]) & planted_ids)]
        if pool:
            clean_questions[cid] = pool
    filler_concepts = sorted(clean_questions)
    n_planted = sum(len(v) for v in PLANTED.values()) + sum(
        len(v) for v in ENDING_STREAK.values()
    )
    n_filler = N_TOTAL - n_planted
    assert n_filler > 0, "植入序列超出总量"

    # 每个填充概念的作答流（概念内保持顺序）
    per = n_filler // len(filler_concepts)
    streams: list[list[tuple[str, int]]] = [
        [(cid, 1 if rng.random() < FILLER_CORRECT_P else 0) for _ in range(per)]
        for cid in filler_concepts
    ]
    # 余数随机分配
    for _ in range(n_filler % len(filler_concepts)):
        cid = rng.choice(filler_concepts)
        for stream in streams:
            if stream and stream[0][0] == cid:
                stream.append((cid, 1 if rng.random() < FILLER_CORRECT_P else 0))
                break

    merged: list[tuple[str, int]] = []
    pending = {cid: list(seq) for cid, seq in PLANTED.items()}
    filler_by_cid: dict[str, list[int]] = defaultdict(list)
    for s in streams:
        for cid, ok in s:
            filler_by_cid[cid].append(ok)

    # 交错：每一拍以概率挑一个"还有货"的流
    while any(pending.values()) or any(filler_by_cid.values()):
        available = [cid for cid, seq in pending.items() if seq] + [
            cid for cid, seq in filler_by_cid.items() if seq
        ]
        cid = rng.choice(available)
        if cid in pending and pending[cid]:
            merged.append((cid, pending[cid].pop(0)))
        else:
            merged.append((cid, filler_by_cid[cid].pop(0)))

```

```
    # 末尾追加"连续错"窗口（rotate + delete，时间连续）
    streak = [(cid, ok) for cid, seq in ENDING_STREAK.items() for ok in seq]
    merged.extend(streak)
    assert len(merged) == N_TOTAL, f"expected {N_TOTAL}, got {len(merged)}"

    t = datetime(2026, 10, 12, 9, 0, 0)
    rows = []
    for cid, ok in merged:
        pool = clean_questions.get(cid) or questions[cid]
        q = rng.choice(pool)
        t += timedelta(minutes=rng.randint(8, 50))
        rows.append(
            {"user_id": "demo-user", "question_id": q["id"], "correct": bool(ok),
             "ts": t.isoformat()}
        )

    out = ROOT / "data/demo_attempts.jsonl"
    with open(out, "w", encoding="utf-8") as f:
        for r in rows:
            f.write(json.dumps(r, ensure_ascii=False) + "\n")

    # 自检输出
    per_concept: dict[str, list[int]] = defaultdict(list)
    qmap = {q["id"]: q for q in all_q}
    for r in rows:
        for cid in qmap[r["question_id"]]["concepts"]:
            per_concept[cid].append(int(r["correct"]))
    print(f"written {out} rows={len(rows)} users=1")
    for cid in sorted(planted_ids):
        seq = per_concept[cid]
        print(f"  {cid:22s} attempts={len(seq):2d} acc={sum(seq)}/{len(seq)}")


if __name__ == "__main__":
    main()

# ============ 文件: /Users/xi/Documents/exam-diag/scripts/make_demo_attempts_multi.py （129 行） ============
"""生成 data/demo_attempts_multi.jsonl：3 个用户、三条不同方向的薄弱链。

  demo-a (408)   树遍历 → AVL：traverse 先对后连错，rotate/delete 全错，前置掌握
  demo-b (408)   进程调度 → 死锁：sched 全错、deadlock 几乎全错，process/sync 掌握
                 （图谱里 deadlock 的前置是 sync 而非 sched，诊断会给出两个根因，调度更弱排第一）
  demo-c (math1) 期望 → 方差：expectation 弱、variance 全错，discrete/cont 掌握
                 （图谱没有"条件期望"节点，链取真实前置边 variance ← expectation；
                 cond 保持基本掌握，避免衍生多余根因）

每人 150-200 条（植入 20-36 条 + 填充 ~140 条、80% 正确率）。seed 固定可复现。
运行：python scripts/make_demo_attempts_multi.py
"""
```

```
from __future__ import annotations

import json
import random
from collections import defaultdict
from datetime import datetime, timedelta
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
SEED = 20261101
FILLER_CORRECT_P = 0.80

PLANTED: dict[str, dict[str, list[int]]] = {
    "demo-a": {
        "ds.stack.stack": [1, 1, 1, 1],
        "ds.recursion": [1, 1, 1, 1],
        "ds.tree.binary": [1, 1, 1, 1, 0],
        "ds.bst.bst": [1, 0, 1, 1, 0],
        "ds.tree.traverse": [1, 1, 1, 0, 0, 0, 0, 0],
        "ds.bst.rotate": [0, 0, 0, 0, 0, 0],
        "ds.bst.delete": [0, 0, 0, 0],
    },
    "demo-b": {
        "os.process.process": [1, 1, 1, 1, 1],
        "os.process.sync": [1, 1, 1, 1],
        "os.process.sched": [0, 0, 0, 0, 0, 0],
        "os.process.deadlock": [0, 1, 1, 0, 0, 0],
    },
    "demo-c": {
        "math.prob.discrete": [1, 1, 1, 1],
        "math.prob.cont": [1, 1, 1, 1, 1],
        "math.prob.cond": [1, 1, 1, 1, 0],
        "math.prob.expectation": [0, 1, 0, 0, 1, 0],
        "math.prob.variance": [0, 0, 0, 0, 0, 0],
    },
}
SUBJECT_FILES = {"demo-a": "408", "demo-b": "408", "demo-c": "math1"}
TARGET = {"demo-a": 166, "demo-b": 161, "demo-c": 166}


def load_questions() -> dict[str, list]:
    pools: dict[str, list] = defaultdict(list)
    for subj in set(SUBJECT_FILES.values()):
        with open(ROOT / f"data/questions/{subj}.jsonl", encoding="utf-8") as f:
            for line in f:
                if line.strip():
                    q = json.loads(line)
                    for cid in q["concepts"]:
                        pools[cid].append(q)
    return pools
```

```


def main() -> None:
    rng = random.Random(SEED)
    pools = load_questions()
    rows = []

    for user, planted in PLANTED.items():
        planted_ids = set(planted)
        # 填充题池：整题不触碰任何植入概念，避免稀释模式
        clean: dict[str, list] = {}
        for cid, qs in pools.items():
            if cid in planted_ids:
                continue
            ok = [q for q in qs if not (set(q["concepts"]) & planted_ids)]
            if ok:
                clean[cid] = ok
        n_planted = sum(len(v) for v in planted.values())
        n_filler = TARGET[user] - n_planted

        streams = {cid: list(seq) for cid, seq in planted.items()}
        filler_concepts = sorted(clean)
        per_user_filler: dict[str, list[int]] = defaultdict(list)
        for _ in range(n_filler):
            cid = rng.choice(filler_concepts)
            per_user_filler[cid].append(1 if rng.random() < FILLER_CORRECT_P else 0)

        # 多流归并：保持每个概念内部顺序，时间线交错
        merged: list[tuple[str, int]] = []
        while any(streams.values()) or any(per_user_filler.values()):
            avail = [c for c, s in streams.items() if s] + [
                c for c, s in per_user_filler.items() if s
            ]
            cid = rng.choice(avail)
            if streams.get(cid):
                merged.append((cid, streams[cid].pop(0)))
            else:
                merged.append((cid, per_user_filler[cid].pop(0)))
        assert len(merged) == TARGET[user]

        t = datetime(2026, 10, 14, 8, 0, 0)
        for cid, ok in merged:
            # 植入事件的题不得挂其他植入概念，防止综合题把作答泄进别的植入链
            pool = [q for q in pools[cid] if not (set(q["concepts"]) & planted_ids - {cid})]
            q = rng.choice(pool or pools[cid])
            t += timedelta(minutes=rng.randint(8, 45))
            rows.append(
                {"user_id": user, "question_id": q["id"], "correct": bool(ok),
                 "ts": t.isoformat()}
            )
```

```
    out = ROOT / "data/demo_attempts_multi.jsonl"
    with open(out, "w", encoding="utf-8") as f:
        for r in rows:
            f.write(json.dumps(r, ensure_ascii=False) + "\n")

    per_user: dict[str, int] = defaultdict(int)
    for r in rows:
        per_user[r["user_id"]] += 1
    print(f"written {out} rows={len(rows)}")
    for u in sorted(per_user):
        print(f"  {u}: {per_user[u]} attempts")


if __name__ == "__main__":
    main()

# ============ 文件: /Users/xi/Documents/exam-diag/scripts/make_gif.py  (157 行) ============
# ruff: noqa: E501
"""合成 docs/demo.gif：三个用户、三条不同方向的薄弱链 → 三个不同根因。

帧内容全部取自真实子进程输出（绝不手造假终端文本）：
  $ examdiag diagnose --attempts data/demo_attempts_multi.jsonl \
        --concepts data/concepts/ --user demo-a   → 根因 = ds.tree.traverse
  $ ... --user demo-b                             → 根因 = os.process.sched (+死锁)
  $ ... --user demo-c                             → 根因 = math.prob.expectation

终端不清屏，三段依次追加，最后一屏同时呈现 3 个不同根因。
规格：12 fps、总帧数 120-144 (10-12 s) 、表格完整出现后静止 ≥36 帧 (≥3 s) 。
渲染 PIL (Menlo + STHeiti 逐字符回退，CJK 按 2 格宽对齐) ，合成 ffmpeg palette。

运行：python scripts/make_gif.py    (只读 data/，不改 examdiag/)
"""

from __future__ import annotations

import os
import shutil
import subprocess
import tempfile
from pathlib import Path

from PIL import Image, ImageDraw, ImageFont

ROOT = Path(__file__).resolve().parent.parent
OUT = ROOT / "docs/demo.gif"

BG = (13, 17, 23)
FG = (230, 237, 243)
GREEN = (126, 231, 135)
```

```
SIZE = 16
LH = 22
PAD = 24
FPS = 12
USERS = ("demo-a", "demo-b", "demo-c")
EXPECTED_ROOT = {
    "demo-a": "ds.tree.traverse",
    "demo-b": "os.process.sched",
    "demo-c": "math.prob.expectation",
}

FONT_LATIN = "/System/Library/Fonts/Menlo.ttc"
FONT_CJK = "/System/Library/Fonts/STHeiti Medium.ttc"


def load_fonts():
    """CJK 字号取 SIZE+4：STHeiti 全角宽 = 字号，20px 正好等于 Menlo 两格（2×10px），
    与 rich 的 "CJK 记 2 格" 宽度约定一致，表格边线才能对齐。"""
    return (
        ImageFont.truetype(FONT_LATIN, SIZE),
        ImageFont.truetype(FONT_CJK, SIZE + 4),
    )


def needs_cjk_font(ch: str) -> bool:
    """真 CJK（含全角标点）用 STHeiti；ASCII 与 box-drawing/箭头留在 Menlo，
    Menlo 的框线字形是 10px 等宽、能无缝拼成表格边。"""
    return ord(ch) >= 0x2E80


def draw_line(draw, xy, text, fonts, fill=FG):
    font_l, font_c = fonts
    x, y = xy
    for ch in text:
        f = font_c if needs_cjk_font(ch) else font_l
        draw.text((x, y), ch, font=f, fill=fill)
        x += f.getlength(ch)


def text_width(text, fonts):
    font_l, font_c = fonts
    return sum((font_c if needs_cjk_font(ch) else font_l).getlength(ch) for ch in text)


def run_real(user: str) -> list[str]:
    """真实执行 diagnose 子进程，返回 stdout 行；断言预期根因出现在输出里。"""
    env = dict(os.environ, COLUMNS="96", TERM="dumb")
    proc = subprocess.run(
        ["examdiag", "diagnose", "--attempts", "data/demo_attempts_multi.jsonl",
         "--concepts", "data/concepts/", "--user", user],
```

```
        cwd=ROOT, env=env, capture_output=True, text=True, check=True,
    )
    lines = proc.stdout.rstrip("\n").split("\n")
    expected = EXPECTED_ROOT[user]
    assert expected in proc.stdout, f"{user} 的真实输出里必须出现根因 {expected}"
    return lines


def build_frames(fonts) -> list[list[tuple[str, tuple]]]:
    frames: list[list[tuple[str, tuple]]] = []
    terminal: list[tuple[str, tuple]] = []
    type_step, reveal_step = 5, 2

    for idx, user in enumerate(USERS):
        out = run_real(user)
        cmd = (f"examdiag diagnose --attempts data/demo_attempts_multi.jsonl "
               f"--concepts data/concepts/ --user {user}")
        base = [*terminal, ("$ " + cmd, GREEN), ("", FG)]
        for k in range(1, len(cmd) + 1, type_step):
            frames.append([*base, (cmd[:k], FG)])
        frames.append([*base, (cmd, FG)])
        for k in range(2, len(out) + 1, reveal_step):
            frames.append([*base, *(("  " + t, FG) for t in out[:k])])
        terminal = [*base, *(("  " + t, FG) for t in out), ("", FG)]
        frames.append(terminal)
        if idx < len(USERS) - 1:
            frames.append(terminal)  # 段间停顿 1 帧

    frames += [frames[-1]] * 36  # 表格全部出现后静止 ≥3 s
    return frames


def main() -> None:
    fonts = load_fonts()
    frames = build_frames(fonts)
    total_s = len(frames) / FPS
    assert 120 <= len(frames) <= 144, f"帧数 {len(frames)} 不在 120–144"
    assert 10 <= total_s <= 12

    maxw = int(max(text_width(t, fonts) for f in frames for t, _ in f)) + PAD * 2
    maxh = max(len(f) for f in frames) * LH + PAD * 2
    print(f"canvas {maxw}x{maxh}, frames={len(frames)}, ~{total_s:.1f}s")

    tmp = Path(tempfile.mkdtemp())
    for i, fl in enumerate(frames):
        img = Image.new("RGB", (maxw, maxh), BG)
        d = ImageDraw.Draw(img)
        y = PAD
        for text, color in fl:
            draw_line(d, (PAD, y), text, fonts, color)
```

```
            y += LH
        img.save(tmp / f"f{i:04d}.png")

    OUT.parent.mkdir(exist_ok=True)
    ffmpeg = shutil.which("ffmpeg")
    if ffmpeg:
        subprocess.run(
            [ffmpeg, "-y", "-loglevel", "error", "-framerate", str(FPS),
             "-i", str(tmp / "f%04d.png"),
             "-vf", "split[s0][s1];[s0]palettegen=stats_mode=diff[p];[s1][p]paletteuse=dither=bayer:bayer_scale=4",
             "-loop", "0", str(OUT)],
            check=True,
        )
        print("assembled with ffmpeg")
    else:  # pragma: no cover
        ims = [Image.open(p) for p in sorted(tmp.glob("f*.png"))]
        ims[0].save(OUT, save_all=True, append_images=ims[1:],
                    duration=int(1000 / FPS), loop=0, optimize=True)
        print("assembled with PIL fallback")
    shutil.rmtree(tmp)
    print(f"written {OUT} ({OUT.stat().st_size / 1024:.0f} KB), frames={len(frames)}, ~{total_s:.1f}s")


if __name__ == "__main__":
    main()

# ============ 文件: /Users/xi/Documents/exam-diag/scripts/make_manual_assets.py  (167 行) ============
# ruff: noqa: E501
"""软著说明书截图 (Wave 4 任务 4.2a) ：全部来自真实运行，禁止示意。

终端类截图 (PIL 终端风渲染，帧文本 = 真实子进程 stdout，与 docs/demo.gif 同标准) ：
  assets/shot_install.png    pip install -e . + pytest tests/ -q    (安装与自检)
  assets/shot_bkt_fit.png    合成数据 EM 拟合与参数恢复误差 (BKT 模块)
  assets/shot_diagnose.png   examdiag diagnose --user demo-a        (诊断模块)
  assets/shot_remedy.png     三个用户的补习路径 (API 真实输出)
  assets/shot_validate.png   python scripts/validate_data.py all    (数据校验模块)
可视化截图 assets/shot_viz.png 由浏览器实截 viz/report.html (scripts/shot_viz_browser.js) 。

运行：python scripts/make_manual_assets.py
"""

from __future__ import annotations

import os
import subprocess
import textwrap
from pathlib import Path

from PIL import Image, ImageDraw, ImageFont

```

```
ROOT = Path(__file__).resolve().parent.parent
ASSETS = ROOT / "docs/copyright/assets"

BG = (13, 17, 23)
FG = (230, 237, 243)
GREEN = (126, 231, 135)
SIZE = 16
SIZE_CJK = 20
LH = 24
PAD = 22
FONT_LATIN = "/System/Library/Fonts/Menlo.ttc"
FONT_CJK = "/System/Library/Fonts/STHeiti Medium.ttc"


def load_fonts():
    return ImageFont.truetype(FONT_LATIN, SIZE), ImageFont.truetype(FONT_CJK, SIZE_CJK)


def draw_line(draw, x, y, text, fonts, fill=FG):
    fl, fc = fonts
    for ch in text:
        f = fc if ord(ch) >= 0x2E80 else fl
        draw.text((x, y), ch, font=f, fill=fill)
        x += f.getlength(ch)


def render_terminal(title: str, lines: list[tuple[str, str]], out: Path) -> None:
    """lines: (text, color)；首行为命令提示（绿色）。"""
    fonts = load_fonts()
    tmp = Image.new("RGB", (10, 10))
    d = ImageDraw.Draw(tmp)
    maxw = max(
        int(sum((fonts[1] if ord(c) >= 0x2E80 else fonts[0]).getlength(c) for c in t))
        for t, _ in lines
    )
    W = min(max(maxw + PAD * 2, 700), 1500)
    H = len(lines) * LH + PAD * 2 + 30
    img = Image.new("RGB", (W, H), BG)
    d = ImageDraw.Draw(img)
    d.text((PAD, 8), title, font=fonts[0], fill=(120, 130, 145))
    y = 30
    for text, color in lines:
        draw_line(d, PAD, y, text, fonts, color)
        y += LH
    out.parent.mkdir(parents=True, exist_ok=True)
    img.save(out)
    print(f"saved {out} ({W}x{H})")


def run_capture(cmd: list[str], env_cols: str | None = "100") -> list[str]:
```

```
    env = dict(os.environ)
    if env_cols:
        env["COLUMNS"] = env_cols
        env["TERM"] = "dumb"
    proc = subprocess.run(cmd, cwd=ROOT, env=env, capture_output=True, text=True, check=True)
    return proc.stdout.rstrip("\n").split("\n")


def green(cmd: str) -> tuple[str, str]:
    return ("$ " + cmd, GREEN)


def main() -> None:
    # 1) 安装与自检
    install_lines = [green("pip install -e .")]
    install_lines += [("Successfully installed exam-diag-0.1.0", FG)]  # 占位，下面用真实输出
    # 真实执行：安装（幂等）+ pytest
    real_install = run_capture(["pip", "install", "-e", "."])
    real_test = run_capture(["python", "-m", "pytest", "tests/", "-q"])
    install_lines = (
        [green("pip install -e .")]
        + [(t, FG) for t in real_install if t.strip()][-2:]
        + [green("python -m pytest tests/ -q")]
        + [(t, FG) for t in real_test]
    )
    render_terminal("安装与测试自检（真实执行）", install_lines, ASSETS / "shot_install.png")

    # 2) BKT 拟合（真实运行内联脚本）
    fit_script = textwrap.dedent("""
        import numpy as np
        from examdiag.bkt import BKTParams, fit, simulate
        TRUE = BKTParams(p_L0=0.3, p_T=0.1, p_S=0.1, p_G=0.2)
        seqs = list(simulate(TRUE, n_students=2000, seq_len=30, seed=42))
        fitted = fit(seqs, init=BKTParams(p_L0=0.4, p_T=0.15, p_S=0.15, p_G=0.15))
        print("true   :", TRUE.as_array().round(4).tolist())
        print("fitted :", fitted.as_array().round(4).tolist())
        print("abs err:", np.abs(fitted.as_array() - TRUE.as_array()).round(4).tolist())
    """).strip()
    out = run_capture(["python", "-c", fit_script])
    render_terminal(
        "BKT EM 拟合（2000 条合成序列，真实输出）",
        [green("python -c \"fit on 2000 synthetic sequences\"")] + [(t, FG) for t in out],
        ASSETS / "shot_bkt_fit.png",
    )

    # 3) 诊断（demo-a，真实 CLI 子进程）
    diag = run_capture([
        "examdiag", "diagnose", "--attempts", "data/demo_attempts_multi.jsonl",
        "--concepts", "data/concepts/", "--user", "demo-a",
    ])
```

```
    render_terminal(
        "examdiag diagnose --user demo-a（真实输出）",
        [green("examdiag diagnose --attempts data/demo_attempts_multi.jsonl "
               "--concepts data/concepts/ --user demo-a")]
        + [(t, FG) for t in diag if t.strip()],
        ASSETS / "shot_diagnose.png",
    )

    # 4) 补习路径（三个用户，真实 API 输出）
    remedy_script = textwrap.dedent("""
        from examdiag.diagnose import diagnose
        from examdiag.schema import ConceptGraph, load_attempts, load_concepts, load_questions
        merged = {}
        for f in ("408", "math1"):
            merged.update(load_concepts(f"data/concepts/{f}.yaml").concepts)
        concepts = ConceptGraph(concepts=merged)
        questions = (load_questions("data/questions/408.jsonl")
                     + load_questions("data/questions/math1.jsonl"))
        attempts = load_attempts("data/demo_attempts_multi.jsonl")
        for u in ("demo-a", "demo-b", "demo-c"):
            d = diagnose(attempts, questions, concepts, user_id=u)
            r = d.root_causes[0]
            path = " -> ".join(concepts.concepts[c].name for c in r.remedy_path)
            print(f"{u}: 根因 {r.name}({r.concept_id}) mastery={r.mastery:.2f}")
            print(f"   补习路径: {path}")
    """).strip()
    out = run_capture(["python", "-c", remedy_script])
    render_terminal(
        "补习路径输出（三个用户，真实运行）",
        [green("python -c \"print remedy paths for demo-a/b/c\"")] + [(t, FG) for t in out],
        ASSETS / "shot_remedy.png",
    )

    # 5) 数据校验（真实全量校验）
    val = run_capture(["python", "scripts/validate_data.py", "all"], env_cols="110")
    render_terminal(
        "python scripts/validate_data.py all（真实输出）",
        [green("python scripts/validate_data.py all")] + [(t, FG) for t in val],
        ASSETS / "shot_validate.png",
    )


if __name__ == "__main__":
    main()

# ============ 文件: /Users/xi/Documents/exam-diag/scripts/make_notebook.py （72 行） ============
"""生成 examples/demo.ipynb（Wave 2 任务 2.4），随后用 nbclient 试运行验证。"""

from __future__ import annotations

```

```
import nbformat as nbf

nb = nbf.v4.new_notebook()
nb.cells = [
    nbf.v4.new_markdown_cell(
        "# exam-diag demo\n\n"
        "输入一批作答记录，输出**根因概念**（最该补的上游知识点）+ 补习路径 + 7 天复习计划。\n\n"
        "直接 `jupyter lab` / `VS Code` 打开本笔记本从头跑到尾即可，无需联网。"
    ),
    nbf.v4.new_code_cell(
        "from examdiag.diagnose import diagnose\n"
        "from examdiag.schema import load_attempts, load_concepts, load_questions\n"
        "from examdiag.srs import plan_week\n\n"
        "concepts = load_concepts('../data/concepts/408.yaml')\n"
        "questions = load_questions('../data/questions/408.jsonl')\n"
        "attempts = load_attempts('../data/demo_attempts.jsonl')\n"
        "print(f'attempts={len(attempts)} questions={len(questions)} '\n"
        "      f'concepts={len(concepts.concepts)}')"
    ),
    nbf.v4.new_markdown_cell(
        "## 1. 根因诊断\n\n"
        "对每个概念拟合 BKT（EM）得到掌握度，错题沿前置图向上游回溯：\n"
        "前置也未掌握的概念是症状，最上游的破损概念才是根因。"
    ),
    nbf.v4.new_code_cell(
        "diag = diagnose(attempts, questions, concepts)\n\n"
        "for i, r in enumerate(diag.root_causes, 1):\n"
        "    print(f'#{i} {r.name} ({r.concept_id})')\n"
        "    print(f'   掌握度 {r.mastery:.2f} | 错题 {r.wrong_count} | 距离 {r.min_distance}')\n"
        "    print('   补习路径:', ' → '.join(concepts.concepts[c].name for c in r.remedy_path))\n"
        "    explained = '、'.join(concepts.concepts[c].name for c in r.explains)\n"
        "    print('   解释症状:', explained or '—')\n"
        "    print()"
    ),
    nbf.v4.new_markdown_cell(
        "## 2. 未来 7 天复习计划\n\n"
        "间隔与掌握度成反比：掌握度越低，出现越频繁。"
    ),
    nbf.v4.new_code_cell(
        "for p in plan_week(diag):\n"
        "    names = [concepts.concepts[c].name for c in p.concepts]\n"
        "    print(f'Day {p.day}:', '、'.join(names) if names else '（休息）')"
    ),
    nbf.v4.new_markdown_cell("## 3. 全部概念掌握度一览（诊断涉及到的）"),
    nbf.v4.new_code_cell(
        "rows = sorted(diag.mastery.items(), key=lambda kv: kv[1])\n"
        "for cid, m in rows[:8]:\n"
        "    bar = '█' * round(m * 30)\n"
        "    print(f'{concepts.concepts[cid].name:12s} {m:5.2f} |{bar:<30}| {cid}')\n"
        "print(f'... 共 {len(rows)} 个有数据概念')"
    ),
```

```
    ),
    nbf.v4.new_markdown_cell(
        "## 4. 更多\n\n"
        "- `viz/report.html`：浏览器打开，依赖图 + 掌握度着色 + 点击查看涉及错题\n"
        "- `examdiag diagnose --attempts ... --concepts ... --plan --json`：命令行 / JSON 输出\n"
        "- `docs/DATA_FORMAT.md`：三类专业文件的格式说明"
    ),
]
nb.metadata.kernelspec = {
    "display_name": "Python 3", "language": "python", "name": "python3",
}

from pathlib import Path  # noqa: E402

out = Path(__file__).resolve().parent.parent / "examples/demo.ipynb"
out.parent.mkdir(exist_ok=True)
nbf.write(nb, out)
print(f"written {out}")

# ============ 文件: /Users/xi/Documents/exam-diag/scripts/make_report.py  (240 行) ============
"""生成 viz/report.html：单文件知识依赖图可视化（无前端框架）。

数据全部来自真实运行：
  - examdiag.diagnose 在 data/demo_attempts.jsonl 上跑出的 mastery 与根因
  - 概念图谱 prereq 边（方向 前置 -> 后继）

布局：按最长前置链分层（Sugiyama 思路的简化版），Python 端算好坐标直接
吐静态 SVG；JS 只负责点击节点 → 右侧详情（涉及错题）与根因面板。
节点颜色按 mastery 渐变：红(0) → 黄(0.5) → 绿(1)，无数据为灰。

运行：python scripts/make_report.py    （不修改 examdiag/ 与 data/ 的任何文件）
"""

# ruff: noqa: E501  — 文件内嵌 HTML/JS 模板，行宽不受 100 限制

from __future__ import annotations

import json
from collections import defaultdict
from pathlib import Path

from examdiag.diagnose import diagnose
from examdiag.graph import build, topo_order
from examdiag.schema import load_attempts, load_concepts, load_questions

ROOT = Path(__file__).resolve().parent.parent
OUT = ROOT / "viz/report.html"

NODE_W, NODE_H = 152, 44
COL_GAP, ROW_GAP = 46, 18
```

```
MARGIN = 46


def lerp_color(t: float) -> str:
    """mastery 0..1 → 红→黄→绿 渐变。"""
    stops = [(220, 38, 38), (245, 158, 11), (22, 163, 74)]
    seg = min(max(t, 0.0), 1.0) * 2
    i = min(int(seg), 1)
    f = seg - i
    c0, c1 = stops[i], stops[i + 1]
    rgb = tuple(int(a + (b - a) * f) for a, b in zip(c0, c1, strict=True))
    return f"#{rgb[0]:02x}{rgb[1]:02x}{rgb[2]:02x}"


def main() -> None:
    concepts = load_concepts(ROOT / "data/concepts/408.yaml")
    questions = load_questions(ROOT / "data/questions/408.jsonl")
    attempts = load_attempts(ROOT / "data/demo_attempts.jsonl")
    diag = diagnose(attempts, questions, concepts)
    g = build(concepts)

    qmap = {q.id: q for q in questions}
    stats: dict[str, dict] = {}
    for cid in concepts.concepts:
        rows = [a for a in attempts if cid in qmap[a.question_id].concepts]
        wrong_qs, seen = [], set()
        for a in rows:
            if not a.correct and a.question_id not in seen:
                seen.add(a.question_id)
                stem = qmap[a.question_id].stem.replace("\n", " ")[:38]
                wrong_qs.append([a.question_id, stem])
        stats[cid] = {
            "name": concepts.concepts[cid].name,
            "attempts": len(rows),
            "correct": sum(a.correct for a in rows),
            "wrong_questions": wrong_qs,
            "mastery": diag.mastery.get(cid),
        }

    # 分层布局：最长前置链定层，层内按 id 排序
    depth: dict[str, int] = {}
    for cid in topo_order(g):
        ps = concepts.concepts[cid].prereqs
        depth[cid] = 1 + max((depth[p] for p in ps), default=0)
    layers: dict[int, list[str]] = defaultdict(list)
    for cid, d in depth.items():
        layers[d].append(cid)
    pos: dict[str, tuple[float, float]] = {}
    for d in sorted(layers):
        col = layers[d]
```

```
        for i, cid in enumerate(sorted(col)):
            pos[cid] = (
                MARGIN + d * (NODE_W + COL_GAP),
                MARGIN + i * (NODE_H + ROW_GAP),
            )
    n_layers = max(layers) + 1
    max_col = max(len(c) for c in layers.values())
    W = MARGIN * 2 + n_layers * (NODE_W + COL_GAP) - COL_GAP
    H = MARGIN * 2 + max_col * (NODE_H + ROW_GAP) - ROW_GAP

    def esc(s: str) -> str:
        return s.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")

    svg_edges = []
    path_edges = set()
    for r in diag.root_causes:
        for a, b in zip(r.remedy_path, r.remedy_path[1:], strict=False):
            path_edges.add((a, b))
    for u, v in g.edges:
        x0, y0 = pos[u]
        x1, y1 = pos[v]
        mx = (x0 + NODE_W + x1) / 2
        d = f"M {x0 + NODE_W:.0f} {y0 + NODE_H / 2:.0f} C {mx:.0f} {y0 + NODE_H / 2:.0f}, {mx:.0f} {y1 + NODE_H / 2:.0f}, {x1:
        hot = (u, v) in path_edges
        svg_edges.append(
            f'<path class="edge{" hot" if hot else ""}" data-u="{u}" data-v="{v}" d="{d}"/>'
        )

    svg_nodes = []
    for cid, (x, y) in pos.items():
        s = stats[cid]
        m = s["mastery"]
        color = lerp_color(m) if m is not None else "#94a3b8"
        label = esc(s["name"])[:9]
        svg_nodes.append(
            f'<g class="node" data-id="{cid}" transform="translate({x:.0f},{y:.0f})">'
            f'<rect width="{NODE_W}" height="{NODE_H}" rx="10" fill="{color}" fill-opacity="0.92"/>'
            f'<text x="10" y="19" class="nname">{label}</text>'
            f'<text x="10" y="35" class="nid">{esc(cid)[:20]}</text></g>'
        )

    causes = [r.to_dict() for r in diag.root_causes]
    data = {
        "stats": stats,
        "causes": causes,
        "names": {cid: c.name for cid, c in concepts.concepts.items()},
        "prereqs": {cid: c.prereqs for cid, c in concepts.concepts.items()},
    }

    html = TEMPLATE.replace("%DATA%", json.dumps(data, ensure_ascii=False))
```

```
    html = html.replace("%W%", str(W + 340)).replace("%GW%", str(W))
    html = html.replace("%H%", str(H + 150))
    html = html.replace("%EDGES%", "\n".join(svg_edges))
    html = html.replace("%NODES%", "\n".join(svg_nodes))
    html = html.replace("%NC%", str(len(pos)))
    html = html.replace("%NE%", str(len(g.edges)))
    OUT.parent.mkdir(exist_ok=True)
    OUT.write_text(html, encoding="utf-8")
    print(f"written {OUT} ({OUT.stat().st_size / 1024:.0f} KB), "
          f"nodes={len(pos)} edges={len(g.edges)} causes={[r.concept_id for r in diag.root_causes]}")


TEMPLATE = """<!doctype html>
<html lang="zh-CN">
<head>
<meta charset="utf-8">
<title>exam-diag 根因可视化报告</title>
<style>
  body { margin:0; background:#0b1220; color:#e6edf3;
         font-family:"PingFang SC","Hiragino Sans GB",sans-serif; }
  header { padding:14px 20px; border-bottom:1px solid #1f2a3c; }
  header h1 { font-size:18px; margin:0 0 4px; }
  header .cmd { font-family:Menlo,monospace; font-size:12px; color:#7ee787; }
  #wrap { display:flex; }
  #graph { width:%GW%px; overflow:auto; padding:8px; }
  #side { width:320px; padding:14px; border-left:1px solid #1f2a3c; }
  .edge { stroke:#31446b; stroke-width:1.4; fill:none; }
  .edge.hot { stroke:#7ee787; stroke-width:2.6; }
  .node { cursor:pointer; }
  .node:hover rect { stroke:#fff; stroke-width:2; }
  .node.sel rect { stroke:#7ee787; stroke-width:2.6; }
  .nname { fill:#0b1220; font-size:12px; font-weight:600; }
  .nid   { fill:#0b1220; font-size:10px; opacity:.75; font-family:Menlo,monospace; }
  #panel h2, #side h3 { font-size:14px; margin:6px 0; }
  #panel .m { font-family:Menlo,monospace; }
  ul { padding-left:18px; margin:6px 0; } li { font-size:12px; line-height:1.5; }
  .bar { height:8px; border-radius:4px; background:#1f2a3c; margin:6px 0; }
  .bar i { display:block; height:8px; border-radius:4px; }
  .card { background:#111a2c; border:1px solid #1f2a3c; border-radius:8px;
          padding:10px; margin-bottom:10px; }
  small { color:#8b98ab; }
</style>
</head>
<body>
<header>
  <h1>exam-diag 根因可视化报告 (%NC% 概念 / %NE% 依赖边)</h1>
  <div class="cmd">$ examdiag diagnose --attempts data/demo_attempts.jsonl --concepts data/concepts/</div>
</header>
<div id="wrap">
  <div id="graph">
```

```
    <svg width="%GW%" height="%H%" viewBox="0 0 %GW% %H%">
      %EDGES%
      %NODES%
    </svg>
  </div>
  <div id="side">
    <div id="panel" class="card"><h2>点击任意概念节点查看详情</h2>
      <small>颜色 = 掌握度（红 0 → 黄 0.5 → 绿 1，灰 = 无作答数据）；
      绿色加粗路径 = 根因补习路径</small></div>
    <div id="causes"></div>
  </div>
</div>
<script>
const DATA = %DATA%;
const svg = document.querySelector('svg');
const panel = document.getElementById('panel');

function fmtM(m){ return m === null ? '无数据' : (Math.round(m*100)/100).toFixed(2); }

svg.querySelectorAll('.node').forEach(n => {
  n.addEventListener('click', () => {
    svg.querySelectorAll('.node').forEach(x => x.classList.remove('sel'));
    svg.querySelectorAll('.edge').forEach(x => x.classList.remove('hot2'));
    n.classList.add('sel');
    const id = n.dataset.id, s = DATA.stats[id];
    svg.querySelectorAll('.edge').forEach(e => {
      if (e.dataset.u === id || e.dataset.v === id) e.classList.add('hot2');
    });
    const m = s.mastery;
    const bar = m === null ? '' :
      `<div class="bar"><i style="width:${Math.round(m*100)}%;background:${m<0.4?'#dc2626':m<0.7?'#f59e0b':'#16a34a'}"></i></d
    const wq = s.wrong_questions.length
      ? '<ul>' + s.wrong_questions.map(w => `<li><span class="m">${w[0]}</span> ${w[1]}…</li>`).join('') + '</ul>'
      : '<small>无涉及错题</small>';
    const pre = DATA.prereqs[id].map(p => DATA.names[p] + ' (' + p + ')').join('、') || '无（根节点）';
    panel.innerHTML = `<h2>${s.name} <small>${id}</small></h2>`
      + `掌握度：${fmtM(m)}  作答 ${s.attempts} 次 / 对 ${s.correct}${bar}`
      + `<h3>前置概念</h3><small>${pre}</small>`
      + `<h3>涉及错题（${s.wrong_questions.length}）</h3>${wq}`;
  });
});

const c = document.getElementById('causes');
c.innerHTML = '<h3>根因 Top' + DATA.causes.length + '</h3>' + DATA.causes.map((r,i) =>
  `<div class="card"><b>#${i+1} ${r.name}</b> <small>${r.concept_id}</small>`
  + `<div class="m">mastery=${fmtM(r.mastery)} 错题=${r.wrong_count}</div>`
  + `<small>补习路径：${r.remedy_path.map(p=>DATA.names[p]).join(' → ')}</small>`
  + `<br><small>解释症状：${r.explains.map(p=>DATA.names[p]).join('、') || '—'}</small></div>`
).join('');

```

```
// 默认选中第一个根因
const first = DATA.causes[0] && DATA.causes[0].concept_id;
if (first) document.querySelector(`.node[data-id="${first}"]`).dispatchEvent(new Event('click'));
</script>
</body>
</html>
"""

if __name__ == "__main__":
    main()

# ============ 文件: /Users/xi/Documents/exam-diag/scripts/validate_data.py （177 行） ============
"""Wave 0 数据验收脚本。

用法：
  python scripts/validate_data.py concepts data/concepts/408.yaml --subject 408 [--expect 60]
  python scripts/validate_data.py questions data/questions/408.jsonl --subject 408 \
      --concepts data/concepts/408.yaml [--expect 150]
  python scripts/validate_data.py all          # 四科概念 + 四科题目 + 跨文件 id 唯一

退出码非 0 表示有验收项不通过；stdout 逐项打印 PASS/FAIL。
"""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from examdiag.schema import load_concepts  # noqa: E402

SUBJECTS = ("408", "math1", "english", "politics")
ROOT = Path(__file__).resolve().parent.parent

failures: list[str] = []


def check(name: str, ok: bool, detail: str = "") -> None:
    mark = "PASS" if ok else "FAIL"
    print(f"[{mark}] {name}" + (f" — {detail}" if detail else ""))
    if not ok:
        failures.append(name)


def depth_of(graph) -> int:
    """最长前置链长度（层数）。"""
    memo: dict[str, int] = {}
```

```
2851
2852      def d(cid: str) -> int:
2853          if cid in memo:
2854              return memo[cid]
2855          prereqs = graph.concepts[cid].prereqs
2856          memo[cid] = 1 + max((d(p) for p in prereqs), default=0)
2857          return memo[cid]
2858
2859      return max(d(cid) for cid in graph.concepts)
2860
2861
2862  def validate_concepts(path: Path, subject: str, expect: int = 60, strict: bool = True) -> None:
2863      print(f"\n== concepts: {path} ==")
2864      try:
2865          g = load_concepts(path)
2866      except Exception as e:  # noqa: BLE001
2867          check(f"{path}: schema 加载", False, repr(e))
2868          return
2869      check(f"{path}: schema 加载（含 prereq 引用与无环）", True)
2870      check(f"{path}: 节点数 == {expect}", len(g.concepts) == expect, f"actual={len(g.concepts)}")
2871      bad_subject = [cid for cid, c in g.concepts.items() if c.subject != subject]
2872      check(f"{path}: 所有节点 subject == {subject}", not bad_subject, f"bad={bad_subject[:3]}")
2873      roots = g.roots()
2874      dependents: dict[str, int] = dict.fromkeys(g.concepts, 0)
2875      for c in g.concepts.values():
2876          for p in c.prereqs:
2877              dependents[p] += 1
2878      isolated = [cid for cid in g.concepts if not g.concepts[cid].prereqs and dependents[cid] == 0]
2879      if strict:
2880          check(f"{path}: 无孤立节点（根节点均有后继）", not isolated, f"isolated={isolated[:5]}")
2881      else:
2882          check(f"{path}: 根节点数 <= 40", len(roots) <= 40, f"roots={len(roots)}")
2883      d = depth_of(g)
2884      print(f"       stats: nodes={len(g.concepts)} roots={len(roots)} "
2885            f"sinks={sum(1 for v in dependents.values() if v == 0)} depth={d} edges="
2886            f"{sum(len(c.prereqs) for c in g.concepts.values())}")
2887
2888
2889  def validate_questions(
2890      path: Path, subject: str, concepts_path: Path, expect: int = 150, expect_min: int | None = None
2891  ) -> None:
2892      print(f"\n== questions: {path} ==")
2893      g = load_concepts(concepts_path)
2894      ids: list[str] = []
2895      rows = []
2896      ok_json = True
2897      with open(path, encoding="utf-8") as f:
2898          for i, line in enumerate(f, 1):
2899              if not line.strip():
2900                  continue
```

```
            try:
                rows.append(json.loads(line))
                ids.append(rows[-1].get("id", "<missing>"))
            except json.JSONDecodeError as e:
                ok_json = False
                print(f"       line {i}: JSON error {e}")
    check(f"{path}: 每行合法 JSON", ok_json)
    if expect_min is None:
        check(f"{path}: 行数 == {expect}", len(rows) == expect, f"actual={len(rows)}")
    else:
        check(f"{path}: 行数 >= {expect_min} (目标 {expect}) ", len(rows) >= expect_min,
              f"actual={len(rows)}")
    dup_ids = sorted({x for x in set(ids) if ids.count(x) > 1})
    check(f"{path}: id 无重复", not dup_ids, f"dups={dup_ids[:5]}")
    bad_subject = [r.get("id") for r in rows if r.get("subject") != subject]
    check(f"{path}: 所有题 subject == {subject}", not bad_subject, f"bad={bad_subject[:3]}")
    unknown = [
        (r.get("id"), cid)
        for r in rows
        for cid in r.get("concepts", [])
        if cid not in g.concepts
    ]
    check(f"{path}: concepts 全部可解析", not unknown, f"unknown={unknown[:5]}")
    no_concept = [r.get("id") for r in rows if not r.get("concepts")]
    check(f"{path}: 每题 >=1 个 concept", not no_concept, f"empty={no_concept[:5]}")
    no_source = [r.get("id") for r in rows if not str(r.get("source", "")).strip()]
    check(f"{path}: source 非空", not no_source, f"empty={no_source[:5]}")


def validate_all() -> None:
    for subject in SUBJECTS:
        validate_concepts(ROOT / f"data/concepts/{subject}.yaml", subject,
                          strict=(subject != "english"))
    seen: dict[str, str] = {}
    dups: list[str] = []
    for subject in SUBJECTS:
        path = ROOT / f"data/concepts/{subject}.yaml"
        if not path.exists():
            continue
        g = load_concepts(path)
        for cid in g.concepts:
            if cid in seen:
                dups.append(f"{cid} in {seen[cid]} & {subject}")
            else:
                seen[cid] = subject
    check("跨文件 concept id 唯一", not dups, f"dups={dups[:5]}")
    for subject in SUBJECTS:
        qpath = ROOT / f"data/questions/{subject}.jsonl"
        cpath = ROOT / f"data/concepts/{subject}.yaml"
        if qpath.exists() and cpath.exists():
```

```
            # SPEC：超时时 408/math1 必须满 150，english/politics 可降级到 60
            if subject in ("408", "math1"):
                validate_questions(qpath, subject, cpath, expect=150)
            else:
                validate_questions(qpath, subject, cpath, expect=150, expect_min=60)


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    sub = parser.add_subparsers(dest="cmd", required=True)
    pc = sub.add_parser("concepts")
    pc.add_argument("path", type=Path)
    pc.add_argument("--subject", required=True)
    pc.add_argument("--expect", type=int, default=60)
    pq = sub.add_parser("questions")
    pq.add_argument("path", type=Path)
    pq.add_argument("--subject", required=True)
    pq.add_argument("--concepts", required=True, type=Path)
    pq.add_argument("--expect", type=int, default=150)
    sub.add_parser("all")
    args = parser.parse_args()

    if args.cmd == "concepts":
        validate_concepts(args.path, args.subject, args.expect, strict=(args.subject != "english"))
    elif args.cmd == "questions":
        validate_questions(args.path, args.subject, args.concepts, args.expect)
    else:
        validate_all()

    print()
    if failures:
        print(f"RESULT: {len(failures)} check(s) FAILED: {failures}")
        return 1
    print("RESULT: all checks PASSED")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())

```